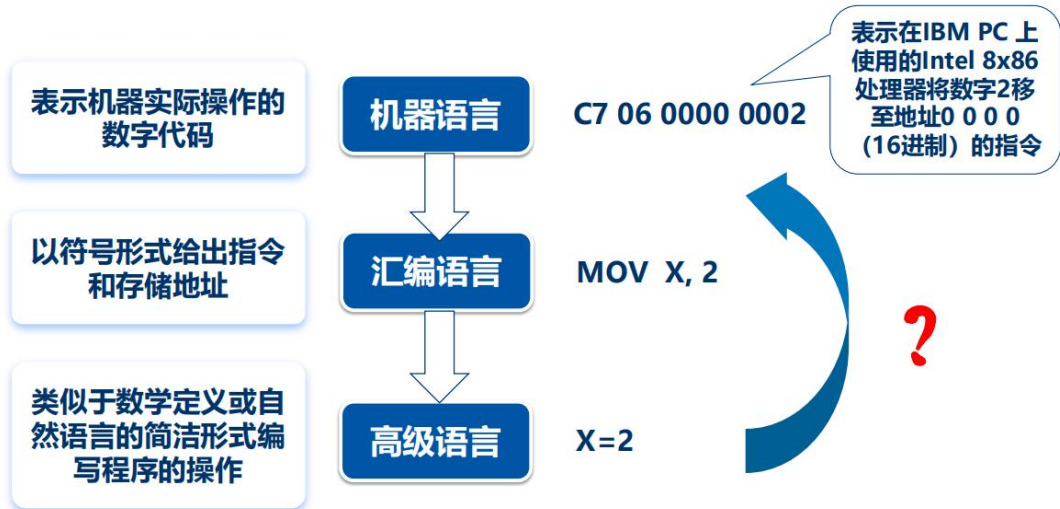


一、绪论：【小题】

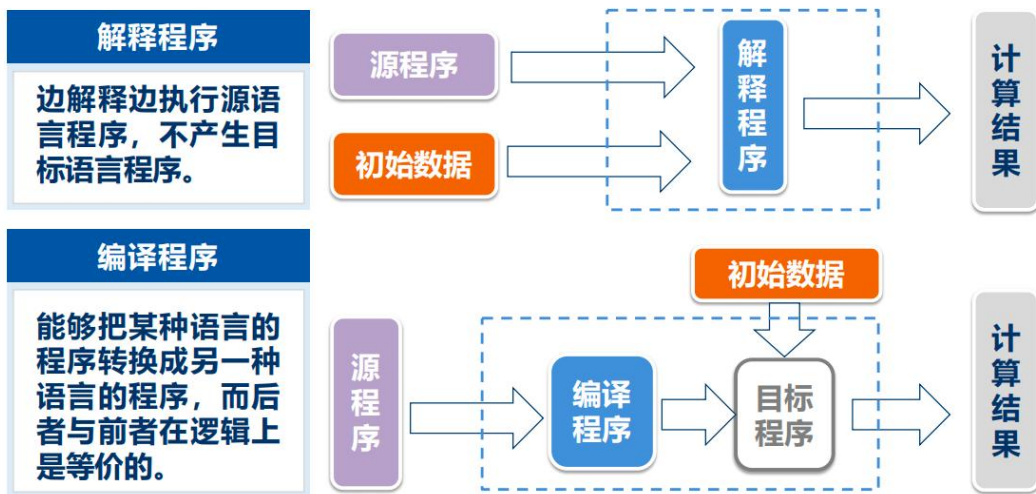
程序语言技术的发展



1. 编译程序&解释程序

程序的执行方式

■ 高级语言程序通常采用解释方式和编译方式两种方式执行



- 编译程序：源程序的转换系统、得到源程序的目标代码、实现上把中间代码转换成目标程序、对整个程序进行分析，翻译成等价机器语言程序后执行；eg.Pascal、Fortran、C；优点—执行效率高；缺点—在运行中发现的错误必须查找整个程序确定、移植性不好；
- 解释程序：源程序的执行系统、得到源程序的执行结果、实现上是执行中间代码、逐个语句地分析和执行；eg.Basic、Python；优点—易于差错、启动速度快、内存使用少、跨平台移植性好；缺点—效率低、运行速度慢；

2. 编译过程（5个）

词法分析、语法分析、语义分析中间代码生成、优化、目标代码生成

<1>词法分析：“扫描程序”—读源程序的字符流、识别单词，并转换成内部形式

输入：源程序中的字符流

输出：等长的内部形式（即属性字）

词法分析依据：语言的词法规则；

描述词法规则的工具：正规式 + 有限自动机 【状态图、DFA、NFA】

编译程序的结构



<2>语法分析：“识别程序”——读入由词法分析程序识别出的符号，根据给定的语法规则，识别出各个语法单位，并生成另一种内部表示

输入：词法分析识别并转换出的符号

输出：另一种内部表示，eg 语法分析树 or 其他中间表示

语法规则描述：上下文无关文法

具体方法：递归子程序、LR 分析、算符优先分析

<3.1>语义分析：对语法树 or 其他内部中间表示进行静态语义检查；如果正确则进行中间代码的翻译

- 按照语法树的层次关系和先后次序，逐个语句地进行语义处理；
- 进行类型审查，审查每个算符是否符合语言规范：类型匹配/类型转换

<3.2>中间代码：独立于具体硬件的记号系统，但能比较容易地变换成机器指令；将各类语法单位，如“表达式”、“语句”、“程序”等翻译为中间代码序列。

输入：句子

输出：中间代码序列

中间代码形式：四元式、三元式、逆波兰式

方法：DAG 图、语法制导翻译、语义子程序

<4>优化：

输入：前段产生的中间代码

输出：功能相同，但更高效的中问代码

原则：程序的等价变换

方法：循环优化、公共子表达式提取、删除无用代码等

<5>目标代码生成

输入：中间代码

输出：特定机器上的低级语言代码—依赖具体的硬件系统结构和机器指令含义！（与机器相关）

<6>表格管理：符号表（最重要）、常数表、标号表、入口名表、过程引用表、循环表、等价名表、公用链表、格式表、中间代码表

符号表：记录源程序中使用的标识符、收集每个标识符的属性信息—类型、作用域、分配存储信息

<7>出错处理：语法错误（词法分析和语法分析阶段）+语义错误（编译时 or 运行时）

例如，下列错误都属于语法错误：

(1) 缺少分隔符号：DIMENSION A(10) B(10)

(2) 保留字拼写错误：DEMENSION A(10)

(3) 括号不配对：

WRITE (6,10) (A (1, J), J = 1, 10), I = 1, 10)

此外还有多余分隔符号，无循环终结语句等等。

■ 典型的语义错误：

- 标识符没有说明就使用；
- 标号有引用而无定义；
- 形式参数和实在参数结合时在类型、个数、位置等方面不一致等等；

- 这些错误可在编译时查出来，而另一些错误如下标越界、运算溢出、调用某些标准函数时自变量的值不符合要求等，则需要到程序运行时才能查出来。

43

，可以将各部分组合成前端和后端。

编译过程

前端：词法分析、语法分析、语义分析、中间代码生成、优化工作。

后端：目标代码生成、出错处理、符号表操作。

主要与源语言有关

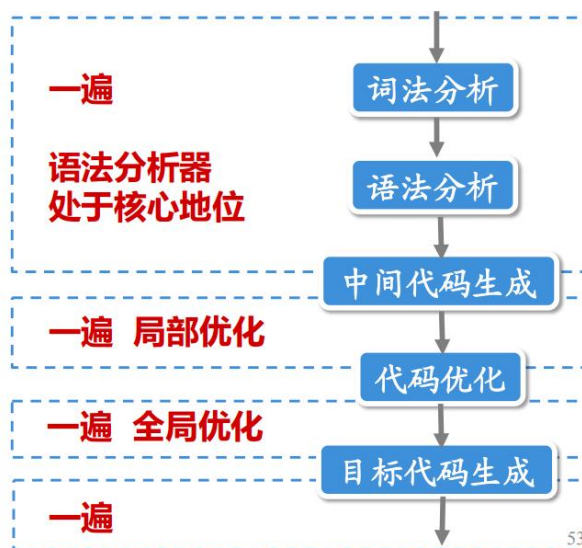
主要与目标代码有关

3. 一遍扫描与多遍扫描（Pass）

- 一个编译程序可由一遍、两遍或多遍完成。每一遍可完成不同阶段或多个阶段的工作。
- 多遍编译：少占内存、多耗时间

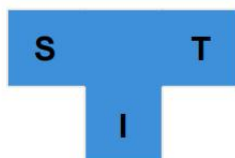
- 一遍编译：多占内存、少耗时间

- 对源程序或源程序的中间结果 **从头到尾扫描一次** 并做相关处理，生成新的中间结果或目标程序的过程。
- “遍”是处理数据的一个完整周期，**每遍工作从外存上获得前一遍的中间结果（源程序），完成它所含的有关工作之后，再把结果记录于外存。**



4. 编译程序生成

- 编译程序实现语言：机器语言、汇编语言（充分发挥各种不同硬件系统的效率、满足各种不同的具体要求）、高级语言（大大节省程序设计的时间、易于阅读维护和移植）



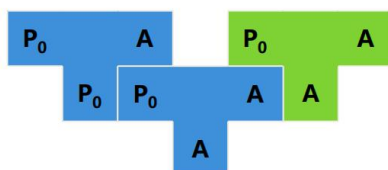
■ 其中：

- S: 源语言(程序), Source language(program)
- T: 目标语言(程序), target/object language(program)
- I: 实现语言, implementation language

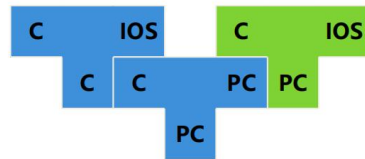
- 编译程序生成技术：自编译（用某种高级语言书写自己的编译程序）、交叉编译、自展、移植

■ 用某种高级语言书写自己的编译程序称为自编译。

- 例如,假定A机器上已有一个PASCAL语言编译程序, 则可用PASCAL语言编写一个功能更强的PASCAL语言编译程序, 然后借助于原有的编译程序对新编写的PASCAL编译程序进行编译, 从而得到一个能在A机器上运行的功能更强的PASCAL编译程序。



- 用x机器上的编译程序产生可在y机器上运行的目标代码称为交叉编译。
- 例如, 若PC机器上已有C语言编译程序,则可用PC机器中的C语言书写一个编译程序, 该编译程序的源程序是C语言程序,而产生的目标程序则是基于IOS终端的, 即产生在IOS终端上执行的低级语言程序。



- 适用于嵌入式系统、移动端开发
- 上述两种方法假定已有一个编译程序, 若没有, 则可采用自展或移植法。

自展

- 首先确定一个非常简单的核心语言L0, 然后用机器语言或汇编语言书写出其编译程序T0;
- 到L1, $L0 \subset L1$, 并用L0编写L1的编译程序T1(自编译);
- 然后再把语言L1扩充为L2, $L1 \subset L2$, 并用L1编写L2的编译程序T2;
-

这样不断扩展, 直到完成所要求的编译程序为止。

- 移植是指A机器上的某种高级语言的编译程序稍加改动后能在B机器上运行。
- 一个程序若能较易地A机移到B机上运行, 则称该程序可移植。

5. 编译程序的分类

- 依据编译程序的不同用途和侧重, 可分类为:
 - 诊断型编译程序
 - 优化型编译程序
 - 交叉型编译程序
 - 可变目标型编译程序

作业题

- 高级程序设计语言有哪两种执行方式? 其特点是什么?
- 什么是编译程序? 它的功能是什么?
- 一个编译程序由哪几个阶段构成?

第一章复习完后自测:

- **基本概念**
 - 编译程序与解释程序的区别
 - 编译包括哪些阶段，每阶段的输入、输出（功能角度）
 - 一遍扫描与多遍扫描的含义、执行过程的区别

二、高级语言及其语法描述

1. 比较

比较	机器语言	汇编语言	高级语言
硬件识别	是唯一可以识别的语言	不可识别	不可识别
是否可直接执行	可直接执行	不可，需汇编、连接	不可，需编译/解释、连接
特点	✓面向机器 ✓占用内存少 ✓执行速度快 ✓使用不方便	➢面向机器 ➢占用内存少 ➢执行速度快 ➢较为直观 ➢与机器语言——对应	■面向问题/对象 ■占用内存大 ■执行速度相对慢 ■标准化程度高 ■便于程序交换，使用方便
定位	低级语言，极少使用	低级语言，很少使用	高级语言，种类多，常用

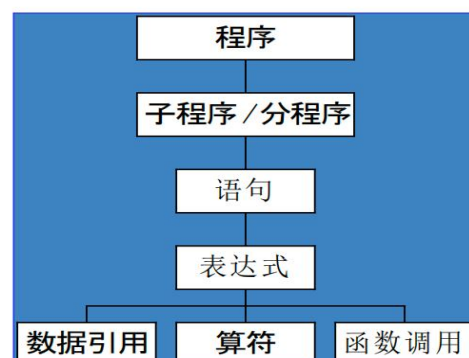
2. 程序语言三大特点—语法、语义、语用



- 语法：词法规则+语法规则
词法规则：单词符号的形成规则：标识符、基本字、常量、算符、界符
语法规则：语法单位的形成规则：表达式、语句、子程序、过程、函数、程序
描述方式：自然语言、语法图、BNF 范式、文法等
- 语义：定义单词符号和语法单位的意义 → 基于属性文法的语法制导翻译

程序语言的功能

- 一个程序语言的基本功能是描述**数据和对数据的运算**。
- 所谓程序，从本质上来说是描述一定数据的处理过程。
- 在现今的程序语言中，一个程序大体可以视为如图所示的层次结构



- 高级语言分类：过程式、函数式、基于规则、面向对象
- 高级语言一般特性：
 - 程序结构：单层结构、多层结构
 - 数据类型与操作：抽象数据类型 ADT—信息隐蔽+数据封装

- 语句与控制结构：说明语句、赋值语句、控制语句、过程调用语句
功能上：执行性语句、说明性语句
形式上：简单句、复合句、子程序等
- 5. 高级语言的语法描述：编译原理=形式语言理论+编译技术
乔姆斯基文法—基本概念

■ 字母表：元素的非空有穷集合。

例：{a, b, c,...,z} (拉丁字母表)

{ α , β , γ ,..., ω } (希腊字母表)

{0,1} (二进制数字字母表)

■ 符号：字母表中的元素。如 a, b,...

■ 符号串：字母表中的符号组成的任何有穷序列。

例. 设字母表 $\Sigma = \{a,b,c\}$

其符号串有：a,b,c,ab,ac,aa,abc,...

符号串与符号组成的顺序有关。

■ 符号串的长度：符号串x中符号的数目, 用|x|表示

■ 空符号串 (空字)：不包含任何符号的符号串, 记为 ϵ 。

■ 符号串的前缀与后缀

- 符号串左部的任意子串, 称为符号串的前缀;
- 符号串右部的任意子串, 称为符号串的后缀。

例. 字母表 $A = \{a,b,c\}$ 上的符号串 $x = ab$, 则x的前缀有:

ϵ 、a、ab;

后缀有:

ϵ 、b、ab。

子符号串

- 设有非空符号串 $u=xvy$ ，则称 v 为符号串 u 的**子符号串**。

例. 符号串 $x=a+b-(c+d)$

则 $a, a+b-, (c+d)$ 等都是 x 的子符号串,

且其长度分别为: $|a|=1,$

$$|a+b-|=4,$$

$$|(c+d)|=5$$

- **符号串集合**: 若集合中所有元素都是某字母表 Σ 上的符号串, 则称之为该字母表上的符号串集合。

➢ 用 Σ^* 表示 Σ 上的所有符号串的全体, 空字也包括在其中。

例. 若 $\Sigma=\{a,b\}$, 则 $\Sigma^*=\{\epsilon, a, b, aa, ab, bb, aaa, \dots\}$ 。

➢ ϕ 表示不含任何元素的**空集** $\{\}$

- 注意区分: ϕ 、 ϵ 、 $\{\epsilon\}$

- Σ^* 的子集 U 和 V 中的 (连接) **积**定义为:

$$UV=\{ \alpha\beta \mid \alpha\in U \ \& \ \beta\in V \}$$

即集合 UV 中的符号串是由 U 和 V 的符号串连接而成的。

例. 设 $U=\{a, b\}$, $V=\{\alpha, \beta, \gamma\}$

则 $UV=\{ a\alpha, a\beta, a\gamma, b\alpha, b\beta, b\gamma\}$

- 注意: 一般 $UV\neq VU$, 但 $(UV)W=U(VW)$ 。

- V自身的n次（连接）积记为：

$$V^n = \underbrace{VVV\dots V}_n$$

- 规定 $V^0 = \{\varepsilon\}$.
- 令： $V^* = V^0 \cup V^1 \cup V^2 \cup \dots$ ，称 V^* 是V的闭包。
- 闭包 V^* 中的每个符号都是由V中的符号串经有限次连接而成的。
- 记 $V^+ = VV^*$ ，称 V^+ 是V的正则闭包。

- 定理. 若A是符号串集合，则 $A^* = (A^*)^*$ 。

证明：显然 $A^* \subseteq (A^*)^*$ ，现证 $(A^*)^* \subseteq A^*$

任给符号串 $x \in (A^*)^*$ ，则存在n，使得 $x \in (A^*)^n$ ，

必存在n个子串 $x_1, \dots, x_n$ ，使得 $x = x_1 \dots x_n$ ，且 $x_i \in A^*$ ($i=1, \dots, n$)。

由 $x_i \in A^*$ ，必存在整数 p_i ，使得 $x_i \in A^{p_i}$ ，

$$\text{令 } M = \sum_{i=1}^n p_i$$

则 $x \in A^M$ ，因此推得 $x \in A^*$ ，

所以 $(A^*)^* \subseteq A^*$ ，

故 $A^* = (A^*)^*$ 。

6. 文法：定义语言的数学模型

- Chomsky文法体系中，任何一种文法必须包含

- 两个不同的有限符号的集合
 - 非终结符集合 V_N
 - 终结符集合 V_T
- 一个起始符S
- 一个形式规则的有限集合 Σ （产生式集合）。

- 注意

- Σ 中的产生式是用来产生语言句子的规则，而句子则是仅由终结符组成的字符串。
- 这些字符串必须从一个起始符S开始，不断使用 Σ 中的产生式而导出来。
- 文法的核心是产生式的集合，它决定了语言中句子的产生。

■ 文法G是一个四元组 $G=(V_N, V_T, S, \mathcal{P})$ ，其中

V_N ：非终结符的有限集合

V_T ：终结符的有限集合， $V_N \cap V_T = \emptyset$

S ：开始符号且 $S \in V_N$ 。

$\mathcal{P}$ ：形式为 $P \rightarrow \alpha$ 的产生式的有限集合，

且 $P \in (V_N \cup V_T)^* V_N (V_N \cup V_T)^*$ ， $\alpha \in (V_N \cup V_T)^*$

■ 文法 $G_1 = (\{N\}, \{0, 1\}, N, \{N \rightarrow 0N, N \rightarrow 1N, N \rightarrow 0, N \rightarrow 1\})$

其中： $V_N = \{N\}$

$V_T = \{0, 1\}$

$S = N$

$\mathcal{P} = \{N \rightarrow 0N, N \rightarrow 1N, N \rightarrow 0, N \rightarrow 1\}$

解释：

非终结符：需要进一步定义的符号，不会出现在程序中。是一个集合，而不是个体记号。

终结符：不需要再定义，会出现在程序中。是组成语言的基本符号。

开始符号 S 至少在某个产生式的左部出现一次。

■ **产生式（规则）**：定义语法单位的一种书写规则

➤ 它的形式为： $P \rightarrow \alpha$ （或 $P ::= \alpha$ ）

其中“ $\rightarrow$ ”读作“定义为”， P, α 为符号串，箭头左边称为产生式左部，箭头右边称为产生式右部。

例： $S \rightarrow 0S1, 0S \rightarrow 01$

➤ 若干个左部相同的产生式如 $P \rightarrow \alpha_1, P \rightarrow \alpha_2, P \rightarrow \alpha_3, \dots,$

$P \rightarrow \alpha_n$ 可合并成一个，缩写为

$P \rightarrow \alpha_1 | \alpha_2 | \alpha_3 | \dots | \alpha_n,$

其中,每个 α_i 称为是 P 的一个**候选式**。

- V_N : 大写字母A、B、C、S等
- V_T : 小写字母, 0~9, +、- 等运算符,
- α, β, γ : 文法符号串, $\in (V_T \cup V_N)^*$
- S: 开始符号, 第一个产生式中出现
- $\rightarrow$: 定义符号 (推出)
- |: 或

- 为了简化, 文法表示
 - 只写出产生式部分
 - 约定第一个产生式的左部符号为初始符号
 - 或
 - 在产生式前写上 "G[A]", 其中G为文法名, A为初始符号.

例. 文法G[N]: $N \rightarrow 0N, N \rightarrow 1N, N \rightarrow 0, N \rightarrow 1$

文法G[E]: $E \rightarrow E+E \mid E^*E \mid (E) \mid i$

从**识别符号**开始, 把当前产生的符号串中的**非终结符号**替换为相应产生式**右部的符号串**, 直到**最终全由终结符号**组成。这种替换过程称为推导或产生句子的过程, 每一步称为**直接推导**或直接产生。

• 规约是推导的逆过程

■ 直接推导

- 如果 $A \rightarrow \gamma$ 是一个产生式, 而 $\alpha, \beta \in (V_T \cup V_N)^*$, 则将产生式 $A \rightarrow \gamma$ 用于符号串 $\alpha A \beta$ 得到符号串 $\alpha \gamma \beta$, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$, 称 **$\alpha A \beta$ 直接推出 $\alpha \gamma \beta$** 。

■ 归约是推导的逆过程

- 若存在 $\alpha A \beta \Rightarrow \alpha \gamma \beta$, 则称 **$\alpha \gamma \beta$ 能够直接归约成 $\alpha A \beta$** 。

■ 推导: 设 $\alpha_1, \alpha_2, \dots, \alpha_n (n > 0) \in (V_T \cup V_N)^*$, 且有

$$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$$

则称这个序列是从 α_1 到 α_n 的一个推导。

■ 若存在从 α_1 到 α_n 的一个推导, 则称 **α_1 可推导出 α_n** 。

■ $\alpha_1 \xRightarrow{+} \alpha_n$ (经一步或若干步推导)

■ $\alpha_1 \xRightarrow{*} \alpha_n$ (经0步或若干步推导)

最左推导对应最右归约:

“最左推导”: 若在推导关系中, 每次最先替换最左的非终结符, 则称为最左推导;

“最右归约”: 若在归约过程中, 每次最先归约最左的非终结符, 则称为最左归约。

- **句型**: 假定G是一个文法, S是它的开始符号,
如果 $S \xRightarrow{*} \alpha$, 则称 α 是文法G的一个句型。

例 $(E+E), (i+E), (i+i), E$ 都是 $G[E]$ 的句型。

- **句子**: 仅由终结符组成的句型称为句子。

例 $(i \times i + i), (i + i)$ 都是 $G[E]$ 的句子。

- **语言**: 文法G所产生句子的全体, 即:

$$L(G) = \{\alpha | S \xRightarrow{*} \alpha, \alpha \in V_T^*\}$$

Chomsky文法体系分类

- 文法 $G = (V_N, V_T, S, \mathcal{P})$,

$\mathcal{P} : P \rightarrow \alpha$, 其中

$$P \in (V_N \cup V_T)^* V_N (V_N \cup V_T)^*,$$

$\alpha \in (V_N \cup V_T)^*$ 属于Chomsky文法体系

- 该体系对产生式的形式做了一些规定, 分为四类,
即0型、1型、2型、3型文法

示例

试对如下语言 $L(G_3) = \{a^n b^n | n \geq 1\}$ 构造文法 G_3 。

解: $L(G_3)$ 由以下一些符号串 x 组成:

$n=1, x=ab$

$n=2, x=aabb$

$n=3, x=aaabbb$

...

$L(G_3) = \{ab, aabb, aaabbb, \dots\}$

由此可见, 集合 $L(G_3)$ 有如下特点:

- 每个符号串呈对称形式, 即 a, b 成对出现;
- $L(G_3)$ 为无穷集合, 描述它的规则中含递归定义;
- 文法中终结符只有 a, b 。

因此可用以下两条产生式定义语言 $L(G_3)$: $S \rightarrow aSb | ab$

即: $G_3 = (\{S\}, \{a, b\}, \{S \rightarrow aSb | ab\}, S)$

7. 乔姆斯基文法体系分类—4类 (0-3型文法) 【分类依据是“产生式的形式”】

3型文法

■ 也称正规文法

➢ 右线性文法 (Right-linear Grammar) :

$A \rightarrow \omega B$ 或 $A \rightarrow \omega$, 其中 $A, B \in V_N, \omega \in V_T^*$ 。

➢ 左线性文法 (Left-linear Grammar) :

$A \rightarrow B\omega$ 或 $A \rightarrow \omega$, 其中 $A, B \in V_N, \omega \in V_T^*$ 。

➢ 对应的语言: 正规语言

➢ 对应的自动机: 有限自动机 (Finite Automaton)。

示例

文法 $S \rightarrow aS, S \rightarrow a$
对应正规式: $a^+, \text{ 或者 } a^*a$

0型文法

■ 0型文法: 无限制文法, 短语文法

➢ 对应的语言: 递归可枚举语言

➢ 与图灵机等价。

示例

下列文法是0型文法

$S \rightarrow aBC|aSBC$
 $CB \rightarrow BC$
 $aB \rightarrow ab$
 $bB \rightarrow bb$
 $bB \rightarrow b$
 $bC \rightarrow bc$
 $cC \rightarrow cc$
 $cC \rightarrow c$

2型文法

■ 也称上下文无关文法 (CFG: Context-free Grammar)

■ 产生式的形式为 $P \rightarrow \alpha$, 其中

$P \in V_N$, 且 $\alpha \in (V_N \cup V_T)^*$

➢ 对应的语言: 上下文无关语言 (CFL: Context-free Language)

➢ 对应的自动机: 下推自动机 (PDA: Pushdown Automaton)。

1型文法

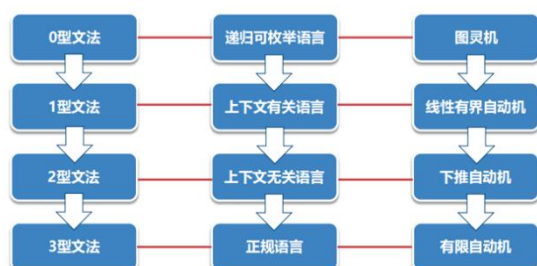
■ 也称上下文有关文法 (CSG: Context-sensitive Grammar)

■ 产生式的形式为 $P \rightarrow \alpha$, 其中

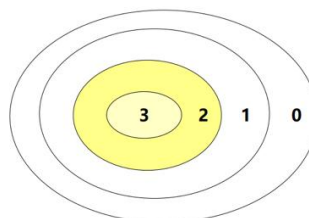
$|P| \leq |\alpha|$, $\alpha \in (V_N \cup V_T)^*$, $P \in (V_N \cup V_T)^* V_N (V_N \cup V_T)^*$

➢ 对应的语言: 上下文有关语言 (CSL: Context-sensitive Language)

➢ 若不考虑 ϵ , 与线性有界自动机 (LBA, Linear Bounded Automaton) 等价



Chomsky文法体系



Conclusion:

<1>自然语言是上下文有关的。

<2>但上下文无关文法有足够的能力描述当前多数程序设计的语法结构。

上下文无关文法描述语言语法结构

■ 算术表达式 的上下文无关文法表示

文法 $G = (\{E\}, \{+, *, i, (,)\}, E, P)$

P: $E \rightarrow i$
 $E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$

■ 条件语句的上下文无关文法表示

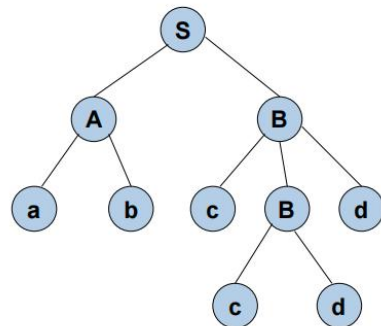
$\langle \text{条件语句} \rangle \rightarrow \text{if} \langle \text{条件表达式} \rangle \text{ then} \langle \text{语句} \rangle$
 $[\text{if} \langle \text{条件表达式} \rangle \text{ then} \langle \text{语句} \rangle \text{ else} \langle \text{语句} \rangle]$

<3>基于**正规文法**讨论**词法分析**问题；基于**上下文无关文法**讨论**语法分析**问题。
其中上下文有关的问题，可通过表格处理解决。

8. 语法树与文法的二义性

■ 语法树是表示一个句型推导过程的图，是一棵倒立的树。

- 结点
- 边
- 根结点
- 末端结点
- 末端分支：A(a,b)和B(c,d)
- 兄弟结点



语法树表明了推导过程中使用了哪条规则和使用在哪个非终结符上，但并没有表明使用规则（产生式）的顺序。

- 如果一个文法的句子（句型）存在两棵语法树,那么,该句子是二义性的。
- 如果一个文法包含二义性的句子,则称这个文法是二义性的; 否则, 该文法是无二义性的。

语言的二义性与文法的二义性问题

- 如语言L找到一个文法是无二义的，则L是无二义的；
- 如未找到一个文法是无二义的，也不能断定L二义。
- 产生某上下文无关语言的每一个文法都是二义的，则称此语言是先天二义的。

例. $\{a^i b^j c^k \mid i, j \geq 1\} \cup \{a^i b^j c^k \mid i, j \geq 1\}$

存在一个二义性的句子 $a^k b^k c^k$

- 文法的二义性是不可判定的。
(因为文法的二义性由句子的语法树决定，不可能对无穷句子来判别)

语法树的特征

■ 给定文法 G , $G=(V_N, V_T, S, \mathcal{L})$, 对于 G 的任何句型都能构造与之关联的语法树(推导树)。这棵树具有下列特征:

- 1、根结点的标记是开始符号 S ;
- 2、每个结点的标记都是 V 中的一个符号;
- 3、若一棵子树的根结点为 A , 且其所有儿子结点的标记从左向右的排列为 $A_1A_2...A_R$, 那么 $A \rightarrow A_1A_2...A_R$ 一定是 $\mathcal{L}$ 中的一条产生式(规则);
- 4、若一标记为 A 的结点至少有一个儿子, 则 $A \in V_N$
- 5、树的叶结点符号所组成的符号串 W 就是所给句型; 若 w 中仅含终结符号, 则 w 为文法 G 所产生的句子。

三、词法分析

1. 词法分析器

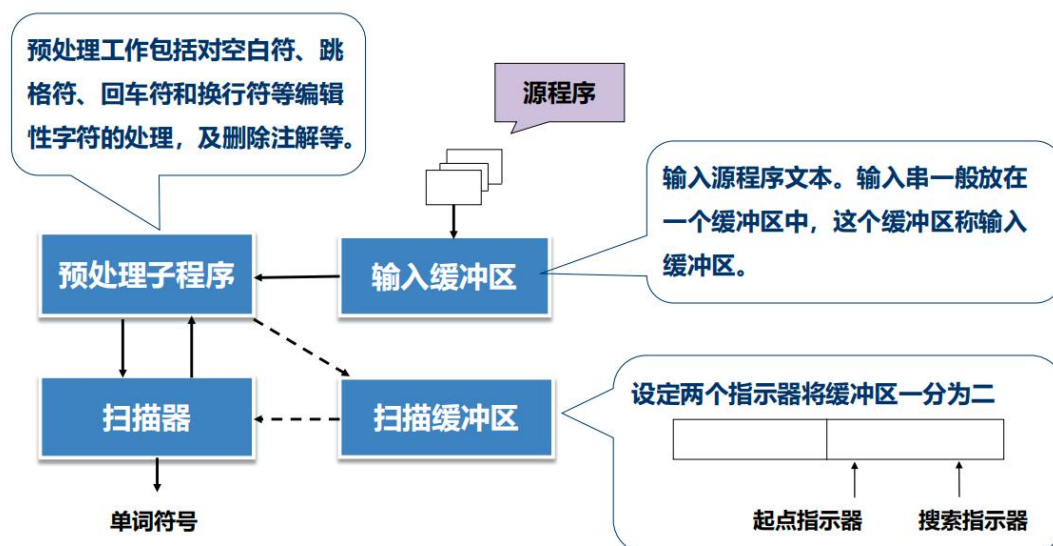
输出: 二元组(单词符号的表示) —— (单词种别, 单词符号的属性值), 前者是语法分析需要的信息, 后者是编译其他阶段需要的信息

⇒ 单词种别: 通常用整数编码——标识符、常数、关键字、运算符、界符;

⇒ 实现方式(2种):

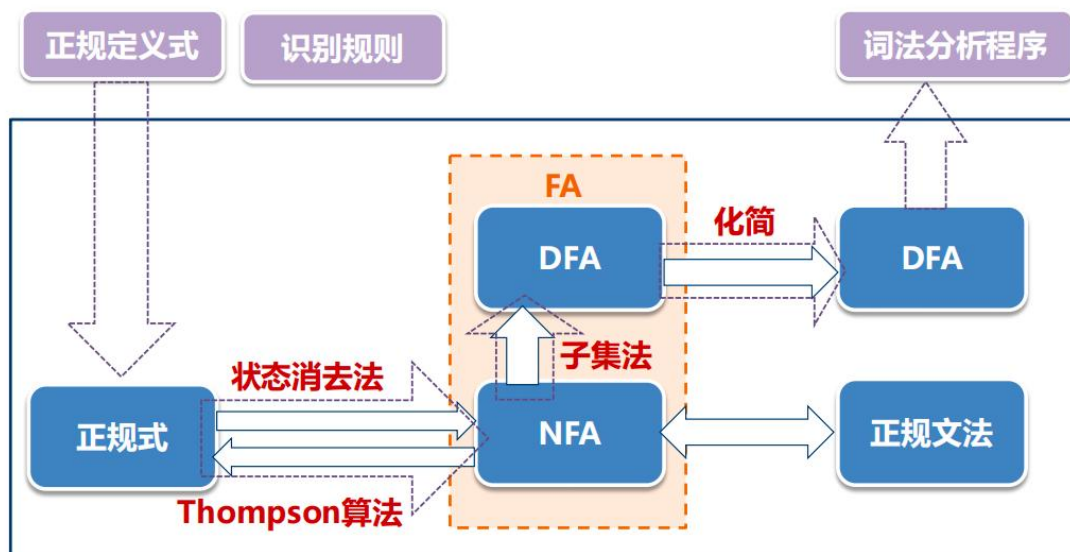
<1>完全独立的方式: 单独一遍实现。词法分析程序读入整个源程序, 它的输出作为语法分析程序的输入;

<2>相对独立的方式: 作为语法分析的一个独立子程序, 语法分析程序需要新符号时调用这个子程序;



2. 正则表达式与有限自动机

【这里概念不太多, 都是大题比较多】



正规式与正规集

■ 字母表 Σ 上的**正规式**和**正规集**递归定义如下:

- (1) ϵ 和 ϕ 都是 Σ 上的正规式, 它们所表示的正规集分别为 $\{\epsilon\}$ 和 ϕ 。
其中: ϵ 为空字符串, ϕ 为空集;
- (2) 任意元素 $a \in \Sigma$, a 是 Σ 上的一个正规式, 它所表示的正规集是 $\{a\}$;
- (3) 假定 U 和 V 都是 Σ 上的正规式, 它们所表示的正规集 记为 $L(U)$ 和 $L(V)$,
那么, $(U|V)$, $(U \cdot V)$ 和 $(U)^*$ 都是正规式,
他们所表示的正规集分别记为 $L(U) \cup L(V)$, $L(U)L(V)$ 和 $(L(U))^*$ 。
- (4) 仅由有限次使用上述三步而得到的表达式才是 Σ 上的正规式
它们所表示的字集才是 Σ 上的正规集。

■ **DFA是一个五元组** $M = (S, \Sigma, \delta, s_0, F)$

- S : 有限的状态集合, 每个元素称为一个**状态**;
 - Σ : 有限的输入字母表, 每个元素称为一个**输入字符**;
 - δ : 转换函数(状态转移集合): $S \times \Sigma \rightarrow S$;
 - s_0 : 初始状态, $s_0 \in S$;
 - F : 终止状态集, $F \subseteq S$;
- 确定有限自动机**

■ 一个非确定的有限自动机 (NFA) M 是一个五元组:

$M = (S, \Sigma, \delta, S_0, F)$, 其中:

(1) S 和 Σ 的定义同前;

(2) $\delta: S \times \Sigma \cup \{\epsilon\} \rightarrow 2^S$ (状态子集);

对于某个状态 $s \in S$ 和一个输入字母 a : $\delta(s, a) = S' \subseteq S$

(3) $S_0 \subseteq S$ 为非空初态集;

(4) $F \subseteq S$ 为终态集

NFA和DFA的比较

■ 初态

- DFA有且仅有一个初态
- NFA可以有多个初态

■ 输入字母

- DFA的每一个状态对于字母表中的每一个符号都有一个转移函数。
- 在NFA中, 一个状态对于字母表中的每一个符号可能不存在转移函数或者存在空转换。

■ 转移状态

- DFA中的下一状态是确定的, 即唯一的
- NFA中的下一状态是不确定的, 可能存在多个转移状态。

NFA和DFA的等价性

■ DFA是NFA的特例, 所以NFA必然能接收DFA能接收的语言。

■ 一个NFA所能接收的语言能被另一个DFA所接收?

■ 设一个NFA接受语言 L , 那么存在一个DFA接受 L 。

- 证明策略: 对于任意一个NFA, 构造一个接收它所能接收语言的DFA, 这个DFA的状态对应了NFA的状态集合。 子集法

二、三章复习完自测：



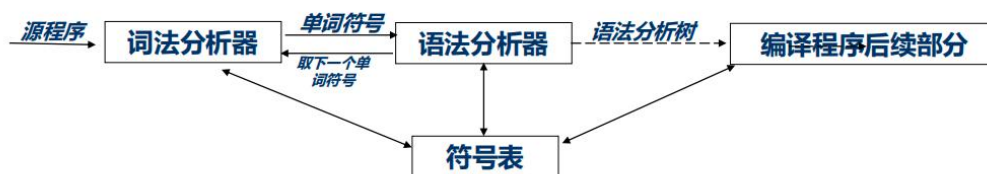
第二、三章（第二章小题、第三章重点！看清题目—自然语言转正则表达式，然后画NFA、DFA、最小化DFA（连贯动作）啥的）

- **文法的基本概念**
 - 文法的定义和分类（四元组、五元组啥的）
 - 语言，句型，句子的概念
 - 句型的推导、语法树与二义性：判定与证明二义性
- **正则表达式：文字描述向正则表达式的转化，正则表达式向NFA的转化**
相关概念
- **NFA确定化为 DFA方法**
- **DFA最小化方法**

四、语法分析——自上而下分析

1. 语法分析器

■ 语法分析器：按照产生式规则，做识别w的工作



语法分析器在编译程序中的地位

2. 自上而下分析：从文法的**开始符号**出发，反复使用各种产生式，寻找与输入符号匹配的最左推导。对任何的输入串(单词符号)，试图用一切可能的办法，从文法的开始符号出发，自上而下地为输入串建立一棵语法树，即为输入串寻找一个最左推导。【就是乱试呗】

带回溯自上而下分析面临的问题

■ 虚假匹配问题

■ 回溯

➢ 回溯会引起时间和空间的大量消耗

■ 文法的左递归问题

➢ 一个文法是含有左递归的，如果存在非终结符P

$$P \Rightarrow^+ P\alpha$$

➢ 含有左递归的文法将使自上而下的分析过程陷入无限循环

■ 报告分析不成功时，难于知道输入串中出错的确切位置。

■ 实际上采用了一种**穷尽一切可能的试探法**，因此效率很低，代价很高

3. LL (1) 分析法：

LL(1)分析法

■ 从左(Left)到右扫描输入串；构造最左(Leftmost)推导；分析时每步向前看一个(1)字符。

■ 目的：构造不带回溯的自上而下分析算法

➢ 左递归的消除

➢ 消除回溯，提左因子

➢ FIRST集合， FOLLOW集合

➢ LL(1)分析条件

➢ LL(1)分析方法

左递归文法

- 一个文法含有下列形式的产生式时,

a) 直接递归

$$A \rightarrow A\beta \quad A \in V_N, \beta \in V^*$$

b) 间接递归

$$A \rightarrow B\beta$$

$$\mathbf{B} \rightarrow \mathbf{A}\alpha \quad \mathbf{A}, \mathbf{B} \in \mathbf{V}_N, \alpha, \beta \in \mathbf{V}^*$$

称为左递归文法。

- 如果一个文法是左递归时，则不能采用自顶向下分析法。

消除左递归的方法:

示例

文法G[S]: $S \rightarrow Qc|c$ $Q \rightarrow Rb|b$ $R \rightarrow Sa|a$

解: 1) 排序: S(1)、Q(2)、R(3)

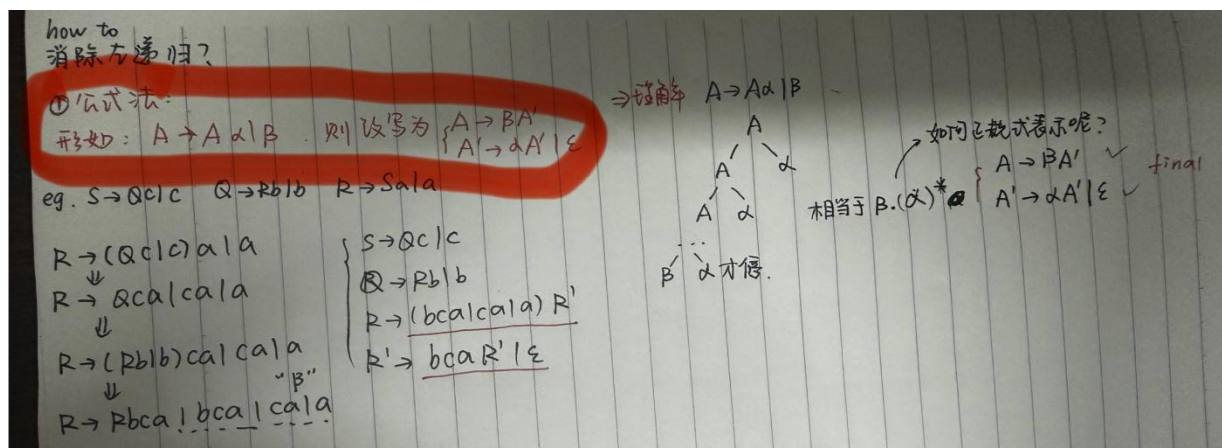
2) 代入得: $S \rightarrow Qc \mid c$

$$Q \rightarrow R \quad b \mid b$$
$$R \rightarrow Rbca \mid bca \mid ca \mid a$$

消除直接左递归:

$$S \rightarrow Qc \mid c$$
$$Q \rightarrow R \quad b \mid b$$
$$R \rightarrow bcaR' \mid caR' \mid aR'$$
$$R' \rightarrow bcaR' \mid \epsilon$$

消除隐含的左递归算法与非终极符排序方法无关



回溯原因

若当前符号 = a ，下一步要展开 A ，

而 $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$ ，

怎样选择 α_i ？

(1) 以 a 为头的 α_i 如果只有一个，则替换唯一；

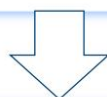
(2) 若以 a 为头有多个 α_i 的，则替换不唯一，可能需要回溯，这是文法的问题，应该变换文法。

回溯解决方法

- 提取公共左因子，将文法改造成任何非终结符的所有候选首符集两两不相交。

$$A \rightarrow \delta\beta_1 | \delta\beta_2 | \dots | \delta\beta_n | \gamma_1 | \gamma_2 | \dots | \gamma_m$$

(其中 $\gamma_1, \gamma_2, \dots, \gamma_m$ 不以 δ 开头)



$$A \rightarrow \delta A' | \gamma_1 | \gamma_2 | \dots | \gamma_m$$

$$A' \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$$

示例

文法 G : $S \rightarrow aSb | aS | \epsilon$

解: 提取: $S \rightarrow aS(\underline{b | \epsilon})$

$S \rightarrow \epsilon$

引入新符: $S \rightarrow aSA | \epsilon$

$\underline{A \rightarrow b | \epsilon}$

一般地，经过反复提取左因子可把每一个非终结符的所有候选首符集变成两两不相交。

4. LL (1) 文法——满足的 3 个条件：目的是进行不带回溯的语法分析

LL (1) 文法

■ 如果文法G满足以下条件：

(1) 文法消除了左递归；

(2) 文法中每个非终结符A的各个产生式的候选首符集两两不相交，即

若 $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$,

则 $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \Phi$, $(i \neq j)$;

(3) 对文法中的每个非终结符A，若它存在某个候选首符集中包含 ϵ ，则 $\text{FIRST}(A) \cap \text{FOLLOW}(A) = \Phi$,

则称该文法G为**LL(1)文法**。

(2) 对每个非终结符的候选式，其任何推导的头符号（终结符）集合两两不相交。

■ 符号串 α 的**终结首符集FIRST (α)** 定义为：

$\text{FIRST}(\alpha) = \{ a \mid \alpha \xRightarrow{*} a\dots, a \in V_T \}$

特别地，若 $\alpha \xRightarrow{*} \epsilon$ ，则规定 $\epsilon \in \text{FIRST}(\alpha)$ 。

■ 以上条件 (2) 可表示为：对文法的任一非终结符号A，若

$A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$

则应有 $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \Phi$, $i \neq j$

FOLLOW

■ 设S是文法G的开始符号，对G的任何非终结符A，定义**A的后继终结符号集**为：

$\text{FOLLOW}(A) = \{ a \mid S \Rightarrow \dots Aa\dots, a \in V_T \}$

■ 特别地,若 $S \Rightarrow \dots A$ ，则规定

$\# \in \text{FOLLOW}(A)$ 。

$\text{FOLLOW}(A)$ 是所有句型中出现在紧接A之后的终结符或“#”。

4.1 FIRST 集求法

符号串 FIRST 集:

FIRST (α) 构造

对于符号串 $\alpha = X_1X_2\ldots X_n$, 构造 FIRST (α)

- (1) 置 $\text{FIRST}(\alpha) = \text{FIRST}(X_1) - \{\epsilon\}$;
- (2) 若对 $1 \leq j \leq i-1$, $\epsilon \in \text{FIRST}(X_j)$, 则把 $\text{FIRST}(X_i) - \{\epsilon\}$ 加到 $\text{FIRST}(\alpha)$ 中;
- (3) 若对 $1 \leq j \leq n$, $\epsilon \in \text{FIRST}(X_j)$, 则把 ϵ 加到 $\text{FIRST}(\alpha)$ 中。

文法符号 FIRST 集

FIRST (X) 构造

对于文法G的每个文法符号 $X \in V_T \cup V_N$, 构造FIRST (X) 的方法是:

- (1) 若 $X \in V_T$, 则 $\text{FIRST}(X) = \{X\}$;
- (2) 若 $X \in V_N$, 且有产生式 $X \rightarrow a\ldots$, $a \in V_T$, 则把 a 加入到FIRST (X) 中, 若有 $X \rightarrow \epsilon$, 则把 ϵ 加入FIRST (X) ;
- (3) 若 $X \in V_N$, 且 $X \rightarrow Y \ldots$, $Y \in V_N$, 则把 $\text{FIRST}(Y) - \{\epsilon\}$ 加到FIRST (X) 中,
若 $X \rightarrow Y_1Y_2 \ldots Y_k$, $Y_1, Y_2, \ldots, Y_{i-1} \in V_N$, $\epsilon \in \text{FIRST}(Y_j)$, 则把
($1 \leq j \leq i-1$)
 $\text{FIRST}(Y_i) - \{\epsilon\}$ 加到FIRST (X) 中。
特别地, 若 $\epsilon \in \text{FIRST}(Y_j)$ ($1 \leq j \leq k$), 则 $\epsilon \in \text{FIRST}(X)$

示例

G: $E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid i$
求每个产生式右部符号串的
FIRST集合

解

$\text{FIRST}(TE') = \{ (, i \}$
 $\text{FIRST}(+TE') = \{ + \}$
 $\text{FIRST}(FT') = \{ (, i \}$
 $\text{FIRST}(*FT') = \{ * \}$
 $\text{FIRST}((E)) = \{ (\}$
 $\text{FIRST}(i) = \{ i \}$

非终结符 FIRST 集:

FOLLOW(A)构造

对于文法G的每个非终结符，构造FOLLOW(A)的方法是:

- (1) 若A为文法开始符号，置#于FOLLOW(A) 中;
- (2) 若有产生式 $B \rightarrow \alpha A \beta$,
则将 $\text{FIRST}(\beta) - \{\epsilon\}$ 加到FOLLOW (A)中;
- (3) 若有 $B \rightarrow \alpha A$ 或 $B \rightarrow \alpha A \beta$, 且 $\beta \xRightarrow{*} \epsilon$
则将FOLLOW(B)加到FOLLOW(A)中;
- (4) 反复使用以上规则, 直至 FOLLOW(A)不再增大为止。

示例

G: $E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid i$

求每个非终结符号的FIRST集合

解

$\text{FIRST}(E) = \text{FIRST}(T)$
 $= \text{FIRST}(F)$
 $= \{ (, i \}$
 $\text{FIRST}(E') = \{ +, \epsilon \}$
 $\text{FIRST}(T') = \{ *, \epsilon \}$

4.2 FOLLOW 集求法【前文已述】

5. 使用 LL (1) 进行语法分析——可以对某个输入串进行有效的无回溯的自上而下分析

■ 设面临的输入符号为a，要用非终结符A进行匹配，且 $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ ，
则可如下分析:

- (1) 若 $a \in \text{FIRST}(\alpha_i)$ ，则指派 α_i 执行匹配任务;
- (2) 否则
 - ① 若 $\epsilon \in \text{FIRST}(A)$ ，且 $a \in \text{FOLLOW}(A)$ ，则让A
与 ϵ 自动匹配;
 - ② 否则，a的出现是一种语法错误。

示例

$G(S): S \rightarrow aA|d, A \rightarrow bAS|d|\epsilon$ 是否为 LL(1) 文法?

解: (1) 不含左递归;

(2) 对于 S , $\text{First}(aA) = \{a\}$ $\text{First}(d) = \{d\}$

对于 A , $\text{First}(bAS) = \{b\}$ $\text{First}(d) = \{d\}$

$\text{First}(\epsilon) = \{\epsilon\}$

任一非终结符的候选首符集两两不相交

满足条件 (2)

(3) $\text{FIRST}(S) = \{a, d\}$ $\text{FIRST}(A) = \{b, d, \epsilon\}$

$\text{Follow}(S) = \{\#, a, d\}$ $\text{Follow}(A) = \{a, d, \#\}$

所以 $\text{FIRST}(A) \cap \text{Follow}(A) = \{d\} \neq \emptyset$

不满足条件 (3)

所以, 该文法不是 LL(1) 文法

示例

如果文法为 $G(S): S \rightarrow aA|d, A \rightarrow bAS|\epsilon$ 是否为 LL(1) 文法?

解: (1) 不含左递归;

(2) 对于 S , $\text{First}(aA) = \{a\}$ $\text{First}(d) = \{d\}$

对于 A , $\text{First}(bAS) = \{b\}$ $\text{First}(\epsilon) = \{\epsilon\}$

任一非终结符的候选首符集两两不相交

满足条件 (2)

(3) $\text{FIRST}(S) = \{a, d\}$ $\text{FIRST}(A) = \{b, \epsilon\}$

$\text{Follow}(S) = \{\#, a, d\}$ $\text{Follow}(A) = \{\#, a, d\}$

满足条件 (3)

所以, 该文法是 LL(1) 文法

求 FIRST 集和 FOLLOW 集的测试题 and 方法:

注意: $[\text{if}]$ 是一个终结符, 同理 $[\text{b}][\text{other}][\text{e}]$

```
1 G(S): S->IETSP|O
2 I->if
3 E->b
4 O->other
5 L->else
6 T->then
7 P->LS|ε
```

求 follow

1 先看开始符

2 看右边A后面有东西 (该东西不能 $\Rightarrow \epsilon$), $\text{First}(\text{东西})$ 加入 $\text{Follow}(A)$, 扣除 ϵ

3 看右边A后面没有东西, $\text{Follow}(\text{左边})$ 加入 $\text{Follow}(A)$

First	Follow
$\text{First}(S) = \{\text{if}, \text{other}\}$	$\text{Follow}(S) = \{\#, \text{else}\}$
$\text{First}(I) = \{\text{if}\}$	$\text{Follow}(I) = \{\text{b}\}$
$\text{First}(E) = \{\text{b}\}$	$\text{Follow}(E) = \{\text{then}\}$
$\text{First}(O) = \{\text{other}\}$	$\text{Follow}(O) = \{\text{else}, \#\}$
$\text{First}(L) = \{\text{else}\}$	$\text{Follow}(L) = \{\text{if}, \text{other}\}$
$\text{First}(P) = \{\text{else}, \epsilon\}$	$\text{Follow}(P) = \{\text{else}, \#\}$
$\text{First}(T) = \{\text{then}\}$	$\text{Follow}(T) = \{\text{if}, \text{other}\}$

```

1 G(E) : E -> TE'
2 E' -> +E | ε
3 T -> FT'
4 T' -> ( | a | b | ^
5 F -> *F'
6 F' -> /F' | ε
7 P -> (E) | a | b | ^

```

求First 第一个推出的是什么
求follow

- 1 先看开始符
- 2 看右边A后面有东西 (该东西不能==>ε), First(东西)加入Follow(A), 扣除ε
- 3 看右边A后面没有东西, Follow (左边) 加入Follow(A)

First	Follow
First(E) = {(, a, b, ^}	Follow(E) = {#, }
First(E') = {+, ε}	Follow(E') = {#, }
First(T) = {(, a, b, ^}	Follow(T) = {+, #, }
First(T') = {(, a, b, ^, ε}	Follow(T') = {+, #, }
First(F) = {(, a, b, ^}	Follow(F) = {(, a, b, ^, +, #,)}
First(F') = {*, ε}	Follow(F') = {(, a, b, ^, +, #,)}
First(P) = {(, a, b, ^}	Follow(P) = {*, (, a, b, ^, +, #,)}

4/4 ✓

6. 递归下降分析:

递归下降分析程序

- **条件**
 - 满足上述LL (1) 文法的条件
- **构成**
 - 一组递归过程
 - 每个递归过程对应G的一个非终结符
- **基本思想**
 - 从文法开始符号出发, 在语法规则(文法产生式)的支配下进行语法分析。逐个扫描源程序中的字符(单词符号), 根据文法和当前输入字符分析到下一个语法成分A时, 便调用识别和分析A的子程序(或其自身), 如此继续下去。

(1) 对每一个非终结符A, 编写一个相应的子程序P(A);

(2) 对于规则 $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ 相应的子程序P(A)构造如下:

```

IF ch IN FIRST( $\alpha_1$ ) THEN P( $\alpha_1$ )
ELSE IF ch IN FIRST( $\alpha_2$ ) THEN P( $\alpha_2$ )
ELSE .....
ELSE IF ch IN FIRST( $\alpha_n$ ) THEN P( $\alpha_n$ )
ELSE IF ( $\epsilon \in \text{FIRST}(A)$ ) AND (ch IN FOLLOW(A))
      THEN RETURN
ELSE ERROR

```


(3) 对于符号串 $\alpha = y_1 y_2 y_3 \dots y_m$, 相应的子程序 $P(\alpha)$ 为:

```
BEGIN  P (y1)
        P (y2)
        ...
        P (ym)
END
```

■ 如果 $y_i \in V_T$, 则 $P(y_i)$ 为:

```
IF ch = yi THEN read(ch)
ELSE ERROR;
```

■ 如果 $y_i \in V_N$, 则 $P(y_i)$ 为上述 (2) 中相应的子程序

示例

$E \rightarrow TE'$; $E' \rightarrow +TE' \mid \varepsilon$; $T \rightarrow FT'$; $T' \rightarrow *FT' \mid \varepsilon$; $F \rightarrow (E) \mid i$

FIRST(+TE') = { + }
FOLLOW(E') = {), # }
FIRST((E)) = { (}

FIRST(*FT') = { * }
FOLLOW(T') = { +,), # }
FIRST(i) = { i }

$E \rightarrow TE'$

```
P(E);
Begin
    P(T); P(E');
End;
```

$E' \rightarrow +TE' \mid \varepsilon$

```
P(E');
Begin
    If ch = '+' Then
        begin
            read(ch);
            P(T); P(E');
        end
    End;
    // ch ∈ Follow(E')?
End;
```

示例

$E \rightarrow TE'$; $E' \rightarrow +TE' \mid \varepsilon$; $T \rightarrow FT'$; $T' \rightarrow *FT' \mid \varepsilon$; $F \rightarrow (E) \mid i$

FIRST(+TE') = { + }
FOLLOW(E') = {), # }
FIRST((E)) = { (}

FIRST(*FT') = { * }
FOLLOW(T') = { +,), # }
FIRST(i) = { i }

$T \rightarrow FT'$

```
P(T);
Begin
    P(F); P(T');
End;
```

$T' \rightarrow *FT' \mid \varepsilon$

```
P(T');
Begin
    If ch = '*' Then
        begin
            read(ch);
            P(F); P(T');
        end
    End;
    // ch ∈ Follow(T')?
End;
```

示例

$E \rightarrow TE'$; $E' \rightarrow +TE' \mid \varepsilon$; $T \rightarrow FT'$; $T' \rightarrow *FT' \mid \varepsilon$; $F \rightarrow (E) \mid i$

$FIRST(+TE') = \{ + \}$

$FIRST(*FT') = \{ * \}$

$FOLLOW(E') = \{ \}, \# \}$

$FOLLOW(T') = \{ +,), \# \}$

$FIRST((E)) = \{ (\}$

$FIRST(i) = \{ i \}$

$F \rightarrow (E) \mid i$

```
P(F);
Begin
  if ch = 'i' then read(ch)
  else if ch = '(' then
    begin
      read(ch); P(E);
      if ch = ')' then
        read(ch)
        else Error
    End
  else Error;
End;
```

示例

$E \rightarrow TE'$; $E' \rightarrow +TE' \mid \varepsilon$; $T \rightarrow FT'$; $T' \rightarrow *FT' \mid \varepsilon$; $F \rightarrow (E) \mid i$

$FIRST(+TE') = \{ + \}$ $FIRST(*FT') = \{ * \}$
 $FOLLOW(E') = \{ \}, \# \}$ $FOLLOW(T') = \{ +,), \# \}$
 $FIRST((E)) = \{ (\}$ $FIRST(i) = \{ i \}$

```
P(E)
BEGIN
  P(T); P(E')
END;
```

```
P(E')
IF ch = " + " THEN
  BEGIN
    read(ch); P(T); P(E');
  END;
ELSE IF ch = " )" THEN
  return;
ELSE IF ch = '#' THEN
  return;
ELSE ERROR;
```

```
P(T)
BEGIN
  P(F); P(T')
END;
```

```
P(T')
IF ch = '*' THEN
  BEGIN
    read(ch); P(F); P(T');
  END;
ELSE IF (ch = '+' OR
ch = ')' OR ch = '#') THEN
  return;
ELSE ERROR;
```

```
P(F)
IF ch = 'i' THEN read(ch);
ELSE IF ch = '(' THEN
  BEGIN
    read(ch); P(E);
    IF ch = ')' THEN read(ch);
  ELSE ERROR
  END
ELSE ERROR;
```

7. 预测分析:

■ 递归下降分析器的局限性

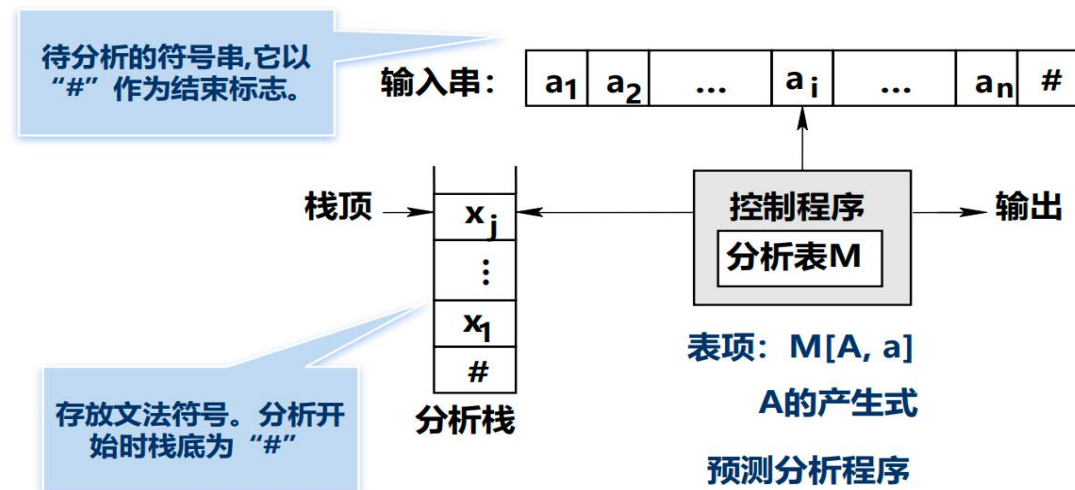
- 需要具有能够实现递归过程的语言和编译系统

■ 预测分析程序

- 使用一个分析表和符号栈进行联合控制, 是实现LL(1)分析的另一种有效方法。

预测分析程序基本思想

- 根据输入串的当前输入符确定选用某一个产生式进行推导，当该输入符与推导的第一个符号相同时，再取输入串的下一个符号，继续确定下一个推导应选的产生式，如此下去，直到推出被分析的输入串为止。
- 预测分析程序 (LL(1)) 分析器组成
 - LL(1)分析表 (预测分析表)
 - 符号栈 (后进先出)
 - 控制程序 (表驱动程序) 组成。



LL (1) 分析表

- 若文法有 m 个非终结符 n 个终结符，则LL (1) 分析表是一个 $(m+1)*(n+2)$ 的矩阵 M
 - 行标题为文法非终结符
 - 列标题为终结符号和输入结束符 #
 - $M[A, a]$ 为一条关于A的产生式，指出当A面临 a 时，应使用的产生式或空格(出错标志)

示例

$E \rightarrow TE' ; E' \rightarrow +TE' \mid \varepsilon ;$

$T \rightarrow FT' ; T' \rightarrow *FT' \mid \varepsilon ;$

$F \rightarrow (E) \mid i$

LL(1)分析表

	i	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

LL(1)分析表的构造

- 预测分析程序中除分析表因文法的不同而不同外，分析栈、控制程序都相同。因此构造一个预测分析程序实际上就是构造文法的LL(1)分析表。

■ 问题

1) 把产生式填到何处?

2) 按 $A \xRightarrow{*} ?$ 将产生式分为两种:

一种是: $A \xRightarrow{*} a...$

另种是: $A \xRightarrow{*} \varepsilon$

6

分析表构造算法

(1) 对每个产生式 $A \rightarrow \alpha$, 执行 (2) 和 (3)

(2) 若 $a \in \text{FIRST}(\alpha)$, 置 $M[A, a] = A \rightarrow \alpha$

(3) 若 $\varepsilon \in \text{FIRST}(\alpha)$, 对 $b \in \text{FOLLOW}(A)$ 置 $M[A, b] = A \rightarrow \varepsilon$

(4) 其余置 $M[A, a] = \text{ERROR}$

总控程序

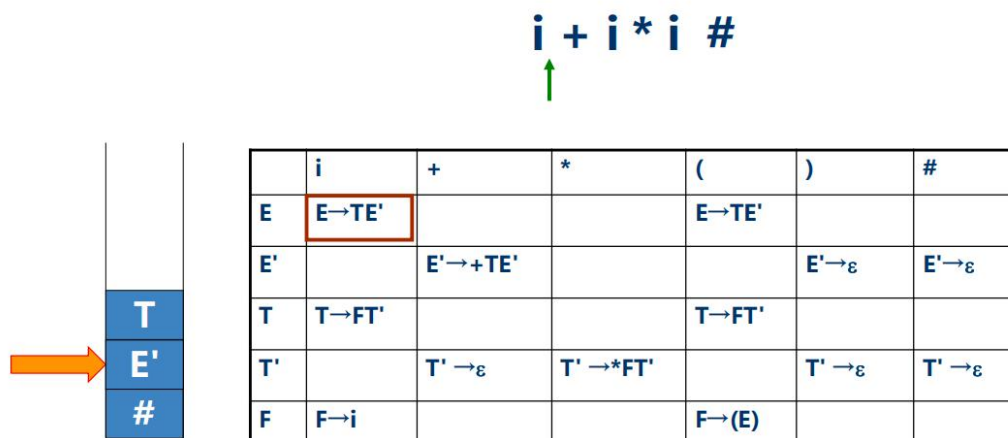
■ 总控程序根据栈顶符号x和当前输入符a，查表决定分析器的动作

(1) 若 $x = a = \text{"\#"}$ ，即STACK 栈顶符号为 $\text{"\#"}$ ，当前输入符号为 $\text{"\#"}$ ，则分析成功。

(2) 若 $x = a \neq \text{"\#"}$ ，即栈顶符号x与当前输入符a匹配，则将x从栈顶弹出，输入串指针后移，读入下一个符号存入a，继续对下一个字符进行分析。

(3) 若x为非终结符A，则查分析表 $M[A, a]$:

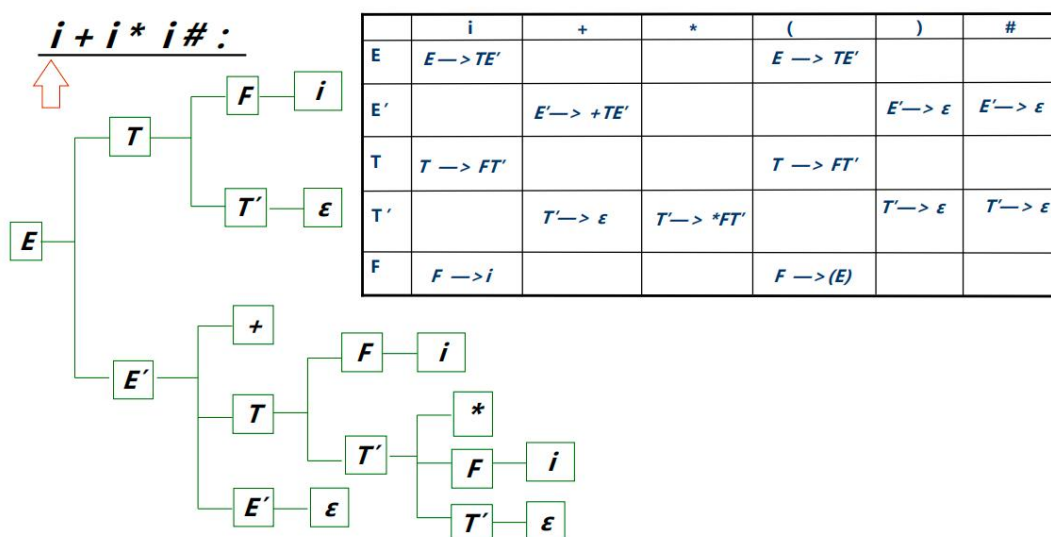
- ① 若 $M[A, a]$ 为一产生式，则A自栈顶弹出， $M[A, a]$ 中产生式的右部符号逆序压栈;
- ② 若 $M[A, a]$ 为 $A \rightarrow \epsilon$ ，则只将A自栈顶弹出。
- ③ 若 $M[A, a]$ 为空，则发现语法错误，调用出错处理程序进行处理。



栈	输入	产生式
0 #E	i+i*i#	
1 #E'T	i+i*i#	$E \rightarrow TE'$
2 #E'T'F	i+i*i#	$T \rightarrow FT'$
3 #E'T'i	i+i*i#	$F \rightarrow i$
4 #E'T'	+i*i#	
5 #E'	+i*i#	$T' \rightarrow \epsilon$
6 #E'T'+	+i*i#	$E' \rightarrow +TE'$
7 #E'T'	i*i#	
8 #E'T'F	i*i#	$T \rightarrow FT'$
9 #E'T'i	i*i#	$F \rightarrow i$
10 #E'T'	*i#	
11 #E'T'F*	*i#	$T' \rightarrow *FT'$
12 #E'T'F	i#	
13 #E'T'i	i#	$F \rightarrow i$
14 #E'T'	#	
15 #E'	#	$T' \rightarrow \epsilon$
16 #	#	$E' \rightarrow \epsilon$

- 输出的产生式就是最左推导的产生式。栈中存放产生式右部，等待与a匹配；
- 当栈顶 $X = a$ 时，分析表指出如何扩充语法树，出错能马上发现。
- 实质：
 - 栈：部分句型，句型右部，还未得到推导的符号。
 - 表：指出VN按哪一条扩充，依赖于VT

上述分析过程生成的语法树:



概述

- 若文法G的预测分析表 $M[A, a]$ 不含有多重定义入口, 当且仅当G为LL(1)文法。
- 文法G是LL(1)的, 则对于G的每一个非终结符A的任何两个不同产生式 $A \rightarrow \alpha \mid \beta$, 有:
 - (1) $FIRST(\alpha) \cap FIRST(\beta) = \varnothing$
 - (2) 若 $\beta \xRightarrow{*} \epsilon$, 则 $FIRST(\alpha) \cap FOLLOW(A) = \varnothing$

小结: 递归下降分析法、LL(1)分析法和预测分析法是编译原理中常见的自顶向下语法分析方法。

- 递归下降分析法是一种自顶向下的语法分析方法，其核心思想是为文法中的每个非终结符编写一个对应的递归函数，函数体根据产生式规则编写，终结符对应消耗输入符号的操作。

通过函数间的递归调用模拟语法树的自上而下构造过程。该方法要求文法是 LL(1)文法，即文法不带回溯且能唯一确定推导路径。

· **LL(1)分析法**是一种不带回溯的非递归自顶向下分析方法，也称为**预测分析法**。它的核心思想是通过当前输入符号和文法符号，唯一确定下一步使用的产生式。其中，“LL”表示从左到右扫描输入串并使用最左推导，“1”表示每次只需查看一个符号即可决定推导方向。

第四章学完自测：

- LL(1)和递归下降分析法是自上而下的分析方法
- **LL(1)分析条件，3个条件（没有左递归、无回溯、First和Follow无交集）**
（分别采取怎样的处理才能满足这三个条件）
- **求解FIRST集和FOLLOW集**
- 求预测分析表
- 熟悉分析过程中栈的变化和所使用的产生式

五、语法分析——自下而上分析

1. 自下而上分析：从输入符号串开始，逐步进行归约，直至归约到文法的开始符号。

- 自下而上分析法就是从输入串开始，逐步进行“归约”，直至归约到文法的开始符号。
- 从语法树的末端，步步向上“归约”，直到根结。

自上而下分析法	开始符号 $S \Rightarrow$ 输入串 α (推导)
自下而上分析法	输入串 $\alpha \Rightarrow$ 开始符号 S (归约)

基本方法：移进-归约法

■ 移进-归约法

使用一个符号栈，把输入符号逐一移进栈，当栈顶形成某个产生式右部时，则将栈顶的这一部分替换（归约）为该产生式的左部符号。

- 实现移进-归约分析的一个方便途径是用一个栈和一个输入缓冲区，用#表示栈底和输入的结束

初始	栈 #	输入串 w#
最终	栈 #S	输入串 #

G: $E \rightarrow E + E \mid E * E \mid (E) \mid i$ 给出 $i_1 * i_2 + i_3$ 的移进归约过程

步骤	栈	输入串	动作
0	#	$i_1 * i_2 + i_3 \#$	预备
1	$\#i_1$	$* i_2 + i_3 \#$	移进
2	$\#E$	$* i_2 + i_3 \#$	归约 $E \rightarrow i$
3	$\#E*$	$i_2 + i_3 \#$	移进
4	$\#E*i_2$	$+ i_3 \#$	移进
5	$\#E*E$	$+ i_3 \#$	归约 $E \rightarrow i$
6	$\#E$	$+ i_3 \#$	归约 $E \rightarrow E * E$
7	$\#E+$	$i_3 \#$	移进
8	$\#E+i_3$	$\#$	移进
9	$\#E+E$	$\#$	归约 $E \rightarrow i$
10	$\#E$	$\#$	归约 $E \rightarrow E + E$
11	$\#E$	$\#$	接受

“移进-归约”是自下而上分析的基本思想，由这个过程可以发现自下而上分析的基本问题是：
<1>如何找出可归约串？

<2>对找出的可归约串替换为哪一个非终结符号？

Solution 则是：算符优先分析、LR 分析。

2. 算符优先分析方法——分析速度快，适用表达式的语法分析

· 定义终结符（算符）的优先关系，按终结符的优先关系控制自下而上的语法分析过程（即寻找可归约串和进行归约）。

- 任何两个可能**相继出现**的终结符a和b（它们之间可能插有一个非终结符）的**优先关系**：

$a < \cdot b$	a的优先级 低于 b
$a \cdot b$	a的优先级 等于 b
$a > b \cdot$	a的优先级 高于 b

注：这三种关系不同于数学中的<,=,>关系。

算符文法判定：

- 一个文法，如果它的任一产生式右部都**不含两个相继（并列）的非终结符**，即不含如下形式的产生式右部：

$\dots QR \dots, \quad Q, R \in V_N$

则称该文法为**算符文法**。

- 设 G为算符文法且不含 ϵ -产生式， $a, b \in V_T$ ，算符间的优先关系定义为：**很好理解呀，就是“谁后推导出来，谁就会先归约，谁的算符优先级就高”**

$a \cdot b$ 当且仅当G含有产生式 $P \rightarrow \dots ab\dots$ 或 $P \rightarrow \dots aQb\dots$

$a < \cdot b$ 当且仅当G含有产生式 $P \rightarrow \dots aR \dots$ 且 $R \xRightarrow{+} \underline{b}\dots$ 或 $R \xRightarrow{+} Qb\dots$

$a > b \cdot$ 当且仅当G含有产生式 $P \rightarrow \dots Rb\dots$ 且 $R \xRightarrow{+} \dots \underline{a}$ 或 $R \xRightarrow{+} \dots aQ$

算符文法！=算符优先文法

- 如果一个算符文法G中的任何终结符对(a, b)至多满足下述关系之一

$a = b$

$a < \cdot b$

$a > \cdot b$

则称 G 为**算符优先文法**。

示例

给定文法G: $E \rightarrow E + E \mid E * E \mid (E) \mid i$ 其中: $V_T = \{+, *, i, (,)\}$ 。

G是算符文法

G是算符优先文法吗?

考察终结符对(+, *)

(1) 因为 $E \rightarrow E + E$, 且 $E \Rightarrow E * E$, 所以

$+ < \cdot *$

(2) 因为 $E \rightarrow E * E$, 且 $E \Rightarrow E + E$, 所以

$+ \cdot > *$

G不是算符优先文法

21

算符优先关系表的构造、FIRSTVT、LASTVT 的求解

■ FIRSTVT (P) 构造

➤ 规则1: 若 $P \rightarrow a \dots$ 或 $P \rightarrow Qa \dots$, 则 $a \in \text{FIRSTVT}(P)$;

➤ 规则2: 若 $a \in \text{FIRSTVT}(Q)$, 且 $P \rightarrow Q \dots$, 则 $a \in \text{FIRSTVT}(P)$ 。

这个也好理解: FIRSTVT意思是第一个推出的终结符;

LASTVT意思是最后一个推出的终结符;

■ LASTVT (P) 构造

➤ 规则1: 若 $P \rightarrow \dots a$ 或 $P \rightarrow \dots aQ$, 则 $a \in \text{LASTVT}(P)$;

➤ 规则2: 若 $a \in \text{LASTVT}(Q)$, 且 $P \rightarrow \dots Q$, 则 $a \in \text{LASTVT}(P)$ 。

二维布尔矩阵 $F[P,a]$ 和符号栈STACK

$$\text{布尔矩阵 } F[P,a] = \begin{cases} .T. & a \in \text{FIRSTVT}(P) \\ .F. & a \notin \text{FIRSTVT}(P) \end{cases}$$

优先关系表的构造

- 通过检查G的每个产生式的每个候选式，可找出所有满足 $a = b$ 的终结符对。

$a = b$

当且仅当G含有产生式 $P \rightarrow \dots ab\dots$ 或 $P \rightarrow \dots aQb\dots$

- 确定满足关系 $<$ 和 $>$ 的所有终结符对：

$a < \cdot b$

当且仅当G含有产生式 $P \rightarrow \dots aR \dots$ 且 $R \xRightarrow{+} b\dots$ 或 $R \xRightarrow{+} Qb\dots$

$a > \cdot b$

当且仅当G含有产生式 $P \rightarrow \dots Rb\dots$ 且 $R \xRightarrow{+} \dots a$ 或 $R \xRightarrow{+} \dots aQ$

- 有了这两个集合之后，就可以通过检查每个产生式的候选式确定满足关系 $<$ 和 $>$ 的所有终结符对。

- 假定有个产生式的一个候选形为

$\dots aP\dots$

那么，对任何 $b \in \text{FIRSTVT}(P)$ ，有 $a < b$ 。

- 假定有个产生式的一个候选形为

$\dots Pb\dots$

那么，对任何 $a \in \text{LASTVT}(P)$ ，有 $a > b$ 。

短语、句型、最左素短语

- 短语

令G是一个文法，S是文法的开始符号，若 $\alpha\beta\delta$ 是文法G的一个句型，如果有

$S \xRightarrow{*} \alpha A \delta$ 且 $A \xRightarrow{+} \beta$

则称 β 是句型 $\alpha\beta\delta$ 相对于非终结符A的短语。



- 素短语

- 是一个短语，它至少含有一个终结符且除它自身之外不含有任何更小的素短语。

- 最左素短语

- 处于句型最左边的那个素短语。

短语：一个句型语法树种的每棵子树（某个结点连同它的所有子孙组成的树）的所有叶子结

点从左到右排列起来形成一个相对于子树根的短语。

示例

设文法G (S) : (1) $S \rightarrow aAcBe$

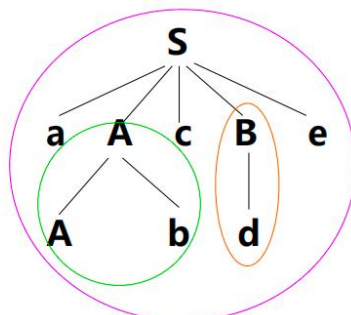
(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$

给出句型aAbcde的短语。

句型aAbcde的语法树为:



短语: d, Ab, aAbcde

示例

对文法G:

(1) $E \rightarrow E+T \mid T$

(2) $T \rightarrow T*F \mid F$

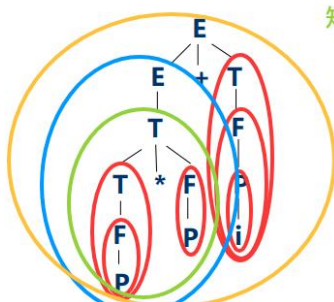
(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

求句型 $P*P+i$ 的短语、素短语、最左素短语

解: 句型的语法树为:

短语: P、i、 $P*P$ 、 $P*P+i$



句型的短语: P, $P*P$, i, $P*P+i$

素短语: $P*P$ i

最左素短语: $P*P$

只要能画出句型的语法树，就不难了。

示例

对文法G:

(1) $E \rightarrow E+T \mid T$

(2) $T \rightarrow T*F \mid F$

(3) $F \rightarrow P \uparrow F \mid P$

(4) $P \rightarrow (E) \mid i$

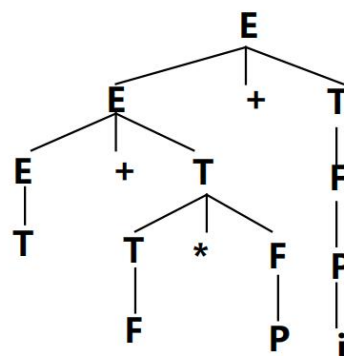
句型: $T+F*P+i$

短语: T, F, P, i, $F*P$,

$T+F*P$, $T+F*P+i$

素短语: $F*P$, i

最左素短语: $F*P$



- 一个算符优先文法G的任何句型的最左素短语是满足下列条件的最左子串 $N_j a_j \dots N_i a_i N_{i+1}$

$$a_{j-1} < a_j$$

$$a_j = a_{j+1} = \dots = a_{i-1} = a_i$$

$$a_i > a_{i+1}$$

$$\# N_1 a_1 \dots a_{j-1} \overbrace{N_j a_j \dots N_i a_i N_{i+1} a_{i+1} \dots N_n a_n N_{n+1}}^R \#$$

$$< \quad = \quad >$$

算符优先文法考察形式：【例题】求 FIRSTVT、LASTVT、构造算符优先分析表、找一个句型的短语、素短语、最左素短语、给一个句子写出算符优先分析步骤。

示例 对文法G: (1) $E \rightarrow E+T \mid T$ (2) $T \rightarrow T * F \mid F$ (3) $F \rightarrow P \uparrow F \mid P$ (4) $P \rightarrow (E) \mid i$

FIRSTVT						
M =	E	+	*	↑	(	)
	T	T	T	T	F	T
	F	F	T	T	F	T
	P	F	F	F	T	F
LASTVT						
M =	E	+	*	↑	(	)
	T	T	T	T	F	T
	F	F	T	T	F	T
	P	F	F	F	F	T

	+	*	↑	(	)	i
+	>	<	<	<	>	<
*	>	>	<	<	>	<
↑	>	>	<	<	>	<
(	<	<	<	<	=	<
)	>	>	>		>	
i	>	>	>		>	

42

示例 对文法G: (1) $E \rightarrow E+T \mid T$ (2) $T \rightarrow T * F \mid F$
(3) $F \rightarrow P \uparrow F \mid P$ (4) $P \rightarrow (E) \mid i$

句型: $T + F * P + i$

最左素短语: $F * P$

$+ < * > +$

	+	*	↑	(	)	i
+	>	<	<	<	>	<
*	>	>	<	<	>	<
↑	>	>	<	<	>	<
(	<	<	<	<	=	<
)	>	>	>		>	
i	>	>	>		>	

4

示例

对文法G: (1) $E \rightarrow E + T \mid T$ (2) $T \rightarrow T * F \mid F$
(3) $F \rightarrow P \uparrow F \mid P$ (4) $P \rightarrow (E) \mid i$

句子: $i*(i+i)$

对文法G, 符号串 $i*(i+i)$ 的分析过程如下:

符号栈	关系	输入串	最左素短语
#	<.	$i^* (i+i) \#$	
# i	>.	$* (i+i) \#$	i
#N	<.	$* (i+i) \#$	
#N*	<.	$(i+i) \#$	
#N*(	<.	$i+i) \#$	
#N*(i	>.	$+i) \#$	i
#N*(N	<.	$+ i) \#$	
#N*(N+	<.	$i) \#$	
#N*(N+ i	>.	$) \#$	i
#N*(N+N	>.	$) \#$	$N+N$
#N*(N	=.	$) \#$	
#N*(N)	>.	$\#$	(N)
#N*N	>.	$\#$	$N*N$
#N	=.	$\#$	
#N#		成功	

算符优先分析算法

- 1 将输入串依此逐个存入符号栈S中, 直到符号栈顶元素 S_k 与下一个待输入的符号a有优先关系 $S_k > a$ 为止;

符号栈栈顶 优先级最高!

- 2 至此, 最左素短语尾符号 S_k 已在符号栈S的栈顶, 由此往前在栈中找最左素短语的头符号 S_{j+1} , 直到找到第一个 $<$ 为止;

- 3 已找到最左素短语 $S_{j+1} \dots S_k$, 将其归约为某个非终结符N及做相应的语义处理。

44

- 约定任何终结符号有: $a > \#$, $\# < a$
- 在算法的工作过程中, 若出现j减1后的值小于等于0时, 则意味着输入串有错。在正确的情况下, 算法工作完毕时, 符号栈S应呈现:

N

- 由于非终结符对归约没有影响, 因此, 非终结符可以不进符号栈S。

3. 规范归约

短语、直接短语、句柄、素短语、最左素短语【素短语——必须含终结符，是用于算符优先文法，直接短语不是必须含有终结符】

- **短语**：句型语法树中**每棵子树**（某个结点连同它的所有子孙组成的树）的**所有叶子结点从左到右排列起来**形成一个相对于子树根的短语。
- **直接短语**：只有**父子两代的子树**形成的短语。
- **句柄**：语法树中**最左那棵只有父子两代的子树**形成的短语。
- **素短语**
 - 是一个短语，它至少含有一个终结符且除它自身之外不含有任何更小的素短语。
- **最左素短语**
 - 处于句型最左边的那个素短语。

示例

G (E) : $E \rightarrow E+T \mid E-T \mid T$
 $T \rightarrow T * F \mid T / F \mid F$
 $F \rightarrow i \mid (E)$

试找出句型 $(T + i) * i - F$ 的所有短语、直接短语和句柄

解：短语

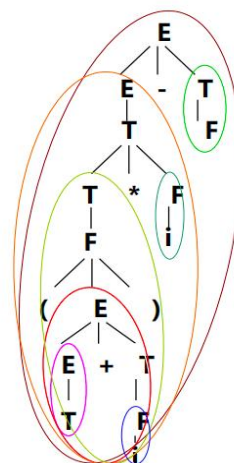
$(T + i) * i - F$
 $(T + i) * i$
 $(T + i)$
 $T + i$
 T
 i (左)
 i (右)
 F

直接短语

T
 i
 i
 F

句柄

T



示例

对文法G: (1) $E \rightarrow E+T \mid T$
 (3) $F \rightarrow P \uparrow F \mid P$

(2) $T \rightarrow T * F \mid F$
 (4) $P \rightarrow (E) \mid i$

句型: $T + F * P + i$

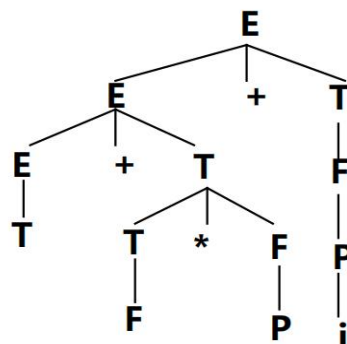
短语: $T, F, P, i, F * P,$
 $T + F * P, T + F * P + i$

直接短语: T, F, P, i

句柄: T

素短语: $F * P, i$

最左素短语: $F * P$



二义性文法的句柄可能不唯一

设 α 是文法G的一个句子, 若序列 $\alpha_n, \alpha_{n-1}, \dots, \alpha_0$, 满足:

(1) $\alpha_n = \alpha$;

(2) $\alpha_0 = S$;

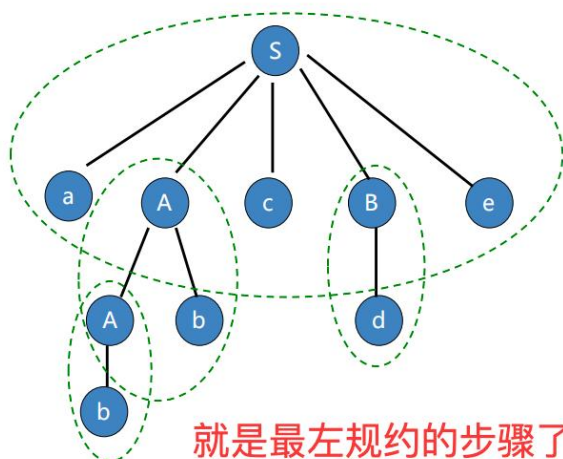
(3) 对任意 $i, 0 < i \leq n$, α_{i-1} 是从 α_i 将句柄替换成相应产生式左部符号而得到的

则称该序列是一个**规范归约**。

规范归约是关于 α 的一个最右推导的逆过程

算符优先分析不是规范归约。

句子abcde 的规范归约过程如下: —— “剪枝”



就是最左规约的步骤了

文法G (S):
 $S \rightarrow aAcBe$
 $A \rightarrow b$
 $A \rightarrow Ab$
 $B \rightarrow d$

规范归约

abbcde

aAbcde

aAcde

aAcBe

S

归约规则

$A \rightarrow b$

$A \rightarrow Ab$

$B \rightarrow d$

$S \rightarrow aAcBe$

最右推导: $S \Rightarrow aAcBe \Rightarrow aAcde \Rightarrow aAbcde \Rightarrow a b b c d e$

最左归约: $a\ b\ b\ c\ d\ e \Rightarrow aAbcde \Rightarrow aAcde \Rightarrow aAcBe \Rightarrow S$

规范归约的基本问题

- 如何找出或确定可归约串——**句柄**？
- 对找出的可归约串——**句柄**替换为哪一个非终结符号？

【算符优先分析是“最左素短语”】

4. LR 分析方法



· LR 分析是严格的“规范归约”！！依旧，不同文法的分析表不同，其余基本一致。

LR分析法

- 在自下而上的语法分析中，算符优先分析算法只适用于算符优先文法，还有很大一类上下文无关文法可以用LR分析法分析。
- LR分析法中的L表示从左向右扫描输入串，R表示构造最右推导的逆。LR分析法是严格的规范归约。就是最左归约.....
- 不足：LR分析法手工构造分析程序工作量相当大。
 - YACC是一个语法分析程序的自动生成器。

LR 分析表：LR (0) 表【基础，有局限性】、SLR 表【简单 LR 表】、规范 LR 表【能力强、代价大】、LALR 表【向前 LR 表，介于 SLR 和规范 LR 之间】

前缀、活前缀

- **前缀**: 一个字的任意首部。例: 字abc的前缀有 $\epsilon, a, ab, \text{或} abc$ 。
- **活前缀**: 规范句型的一个前缀, 前缀的尾符号最多包含到句型的句柄, 即这种前缀不含句柄之后的任何符号 (可归前缀)。
 - 对于规范句型 $\alpha\beta\delta$, β 为句柄, 如果 $\alpha\beta = u_1u_2\dots u_r$, 则符号串 $u_1u_2\dots u_i (1 \leq i \leq r)$ 是 $\alpha\beta\delta$ 的活前缀。(δ 必为终结字符串)

- 在LR分析工作过程的任何时候, 栈里的文法符号 (自栈底向上) 应该构成活前缀。
- 对于一个文法G, 可以构造一个识别G的所有活前缀有限自动机, 并以此构造LR分析表。

LR (0) 项目

- 文法G 的产生式右部加一个圆点($\cdot$), 称为G的一个**LR(0)项目**。它指明了在分析过程的某时刻看到产生式的多大部分。

例. G(E): $E \rightarrow aA$

$A \rightarrow bA|a$

则G的LR (0) 项目有:

$E \rightarrow \cdot aA$ $E \rightarrow a \cdot A$ $E \rightarrow aA \cdot$

$A \rightarrow \cdot bA$ $A \rightarrow b \cdot A$ $A \rightarrow bA \cdot$

$A \rightarrow \cdot a$ $A \rightarrow a \cdot$

拓广文法

- 假定文法G是一个以S为开始符号的文法, 我们构造一个G'
 - 包含整个G;
 - 引进了一个不出现在G中的非终结符S' (G' 的开始符号);
 - 增加一个新产生式 $S' \rightarrow S$ 。
- 称G' 是G的**拓广文法**。
- 拓广文法会有一个仅含项目 $S' \rightarrow S \cdot$ 的状态, 这就是唯一的“接受”态。

示例

文法 $G(S')$

$S' \rightarrow E$

$E \rightarrow aA | bB$

$A \rightarrow cA | d$

$B \rightarrow cB | d$

该文法的项目有：

唯一接受态

- | | | |
|------------------------------|------------------------------|-------------------------------|
| 1. $S' \rightarrow \cdot E$ | 2. $S' \rightarrow E \cdot$ | 3. $E \rightarrow \cdot aA$ |
| 4. $E \rightarrow a \cdot A$ | 5. $E \rightarrow aA \cdot$ | 6. $A \rightarrow \cdot cA$ |
| 7. $A \rightarrow c \cdot A$ | 8. $A \rightarrow cA \cdot$ | 9. $A \rightarrow \cdot d$ |
| 10. $A \rightarrow d \cdot$ | 11. $E \rightarrow \cdot bB$ | 12. $E \rightarrow b \cdot B$ |
| 13. $E \rightarrow bB \cdot$ | 14. $B \rightarrow \cdot cB$ | 15. $B \rightarrow c \cdot B$ |
| 16. $B \rightarrow cB \cdot$ | 17. $B \rightarrow \cdot d$ | 18. $B \rightarrow d \cdot$ |

接下来想要构建 LR (0) 分析表的方法是两个：

<1>构造识别活前缀的 NFA，确定化为 DFA

<2>LR (0) 项目集规范族

但这两个方法其实是等价的，最后得到的 DFA 都是一样的……！

- 构成识别一个文法活前缀的 DFA 的项目集（状态）的全体称为文法的 **LR(0) 项目集规范族**。

- $A \rightarrow \alpha \cdot$ 称为“归约项目”
- 归约项目 $S' \rightarrow \alpha \cdot$ 称为“接受项目”
- $A \rightarrow \alpha \cdot a\beta$ ($a \in V_T$) 称为“移进项目”
- $A \rightarrow \alpha \cdot B\beta$ ($B \in V_N$) 称为“待约项目”。

转换函数 GO 把项目集 I 连接成一个 DFA 转换图

项目集 I 的闭包 CLOSURE(I)

- 假定 I 是文法 G' 的任一项目集，定义和构造 I 的闭包 CLOSURE(I) 如下：

1. I 的任何项目都属于 CLOSURE(I)；
2. 若 $A \rightarrow \alpha \cdot B\beta$ 属于 CLOSURE(I)，那么，对任何关于 B 的产生式 $B \rightarrow \gamma$ ，项目 $B \rightarrow \cdot \gamma$ 也属于 CLOSURE(I)；
3. 重复执行上述两步骤直至 CLOSURE(I) 不再增大为止。

状态转换函数 $GO(I, X)$

- GO 是一个状态转换函数。 I 是一个项目集， X 是一个文法符号。函数值 $GO(I, X)$ 定义为：

$$GO(I, X) = CLOSURE(J)$$

其中

$$J = \{\text{任何形如 } A \rightarrow \alpha X \cdot \beta \text{ 的项目} \mid A \rightarrow \alpha \cdot X \beta \text{ 属于 } I\}.$$

直观上说，若 I 是对某个活前缀 γ 有效的项目集，那么， $GO(I, X)$ 便是对 γX 有效的项目集。

上述都是为了画 LR (0) 有限自动机 DFA 做的准备。

LR0 的考点：拓广文法、LR0 的 DFA、LR0 的分析表 (ACTION、GOTO)、LR0 分析流程

■ LR分析器的核心是一张分析表

- **ACTION[s, a]**: 当状态 s 面临输入符号 a 时，应采取什么动作.
- **GOTO[s, X]**: 状态 s 面对文法符号 X 时，下一状态是什么
 - **GOTO[s, X]**定义了一个以文法符号为字母表的DFA

■ 每一项ACTION[s, a]所规定的四种动作:

1. **移进** 把 (s, a) 的下一状态 s' 和输入符号 a 推进栈，下一输入符号变成现行输入符号.
2. **归约** 指用某产生式 $A \rightarrow \beta$ 进行归约. 假若 β 的长度为 r ，归约动作是：
去除栈顶 r 个项，使状态 s_{m-r} 变成栈顶状态，然后把 (s_{m-r}, A) 的下一状态 $s' = GOTO[s_{m-r}, A]$ 和文法符号 A 推进栈.
3. **接受** 宣布分析成功，停止分析器工作.
4. **报错**

- **LR文法**: 对于一个文法, 如果能够构造一张LR分析表, 使得它的每个入口均是唯一确定, 则该文法称为LR文法。

➢ 在进行自下而上分析时, 一旦栈顶形成句柄, 即可归约。

- **LR(k)文法**: 对于一个文法, 如果每步至多向前检查 k 个输入符号, 就能用LR分析器进行分析。则这个文法就称为LR(k)文法。

➢ 大多数程序语言, 符合LR(1)文法

- **LR(0) 文法**: $k = 0$, 即只要根据当前符号和历史信息进行分析, 而无需展望。

- 假若一个文法G的拓广文法G' 的活前缀识别自动机中的每个状态 (项目集) 不存在下述情况:

(1) 既含移进项目又含归约项目;

(2) 含有多个归约项目

则称G是一个**LR(0)文法**。

当有冲突时, 用“展望”的方式获取上下文信息来解决冲突=》SLR (1)、LR (k)

- 因为只对有冲突的状态才向前查看一个符号, 以确定做哪种动作, 因而称这种分析方法为**简单的LR(1)分析法**, 用**SLR(1)**表示

- **LR(1)状态比SLR多,**

$LR(0) \subset SLR \subset LR(1) \subset \text{无二义文法}$

三个文法的关系: LR0、SLR、LR1:



这一章大概率 LR0 考大题:

列出拓广文法的所有 LR0 项目、构造 LR0 分析表、判断是否为 LR0 文法、给出具体句子的分

析过程。

- 假若一个文法 G 的拓广文法 G' 的活前缀识别自动机中的每个状态（项目集）**不存在**下述情况：

（1） 既含移进项目又含归约项目；

（2） 含有多个归约项目

则称 G 是一个 **LR(0) 文法**。

第五章学完自测

- LR(0)和**算符优先分析**（**不是规范归约**）方法是自底向上的分析方法
- **概念：规范规约、短语、句柄、素短语、最左素短语、活前缀**
- 能够对给定的句型求出短语、句柄、素短语、最左素短语
- 熟悉算符优先文法及其分析方法的基本概念和过程
- **LR(0) 文法的判定**
- **LR(0) 项目集规范族，识别活前缀的DFA**
- **LR(0) 分析表**
- 移进-规约分析过程中栈的变化
- LR(0), SLR, LR(1)之间的关系

【好消息！！你已经学完了最难的部分！……】

六、属性文法和语法制导翻译

1. 形式语义学分类

■ 根据形式化的侧重面和所使用的数学工具的不同，形式语义学可分成：

- **操作语义学**——着重模拟数据加工过程中计算机系统的操作。
- **指称语义学**——主要描述数据加工的结果而不是加工过程的细节。
- **公理语义学**——用公理化的方法描述程序对数据的加工。
- **代数语义学**——把程序设计语言看作是刻画数据和加工数据的一种抽象数据类型，使用研究抽象数据类型的代数方法，来描述程序设计语言的形式语义。

	1	2	3	4
计算模型	图灵机	谓词演算	递归函数论	代数
硬件系统	冯.诺依曼机器		组合逻辑计算机	
程序设计语言	状态语言 Pascal, Ada	逻辑语言 Prolog	函数式语言LISP	类型程序设计语言
形式语义	操作语义	公理语义	指称语义	代数语义

2. 属性文法与语法制导翻译

- **形式语义学方法缺点**：符号系统比较复杂，其描述文本不易读，不能借助这些形式系统自动完成语义处理任务。
- **目前实际应用中比较流行的语义描述和语义处理方法是属性文法和语法制导翻译的方法**

属性文法——语义描述；语法制导翻译——语义处理。

3. 属性文法——基础概念

- 在上下文无关文法的基础上，在描述语义动作时，为每个文法符号（终结符和非终结符）配备若干相关的“值”，如“类型”，“地址”等，称为**属性**。
- 对文法的每个产生式配备一组属性计算规则称为**语义规则**，它的描述形式为 $b:=f(c_1, c_2, \dots, c_k)$ ，其中 $b, c_1, c_2, \dots, c_k$ 为文法符号的属性， f 是一个函数。
- 每个文法符号联系于一组属性，且对每个产生式都给出其语义规则的文法称为**属性文法**。

- 属性代表与文法符号相关信息，如类型、值、代码序列、符号表内容等；
- 属性可以进行计算和传递；
- 在一个属性文法中，对应于每个产生式 $A \rightarrow \alpha$ 都有一组与之相关联的语义规则，每条规则的形式为：

$$b := f(c_1, c_2, \dots, c_k)$$

这里， f 是一个函数

(1) b 是 A 的一个属性，并且 $c_1, c_2, \dots, c_k$ 是产生式右边文法符号的属性，则 b 是 A 的综合属性；

(2) b 是产生式右边某个文法符号 X 的一个属性，并且 $c_1, c_2, \dots, c_k$ 是 A 或产生式右边任何文法符号的属性， b 是 X 的继承属性；

➢ 属性 b 依赖于属性 $c_1, c_2, \dots, c_k$ 。

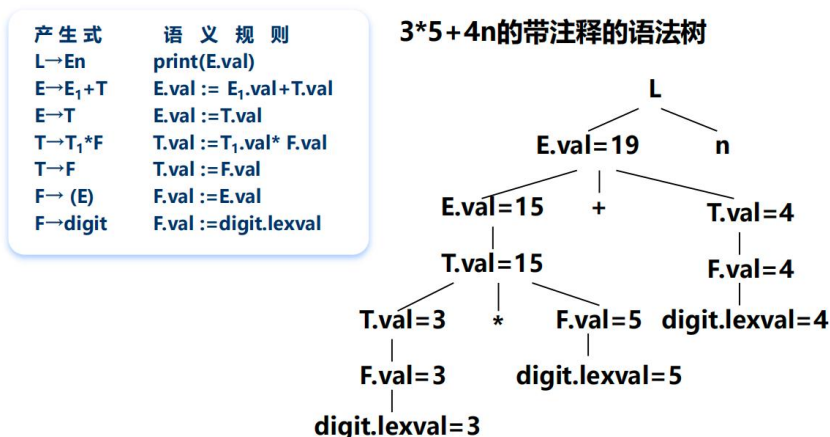
11

- 在语法树中，一个结点的综合属性的值由其子结点的属性值确定。

- 使用自底向上的方法在每一个结点处使用语义规则计算综合属性的值

synathis综合

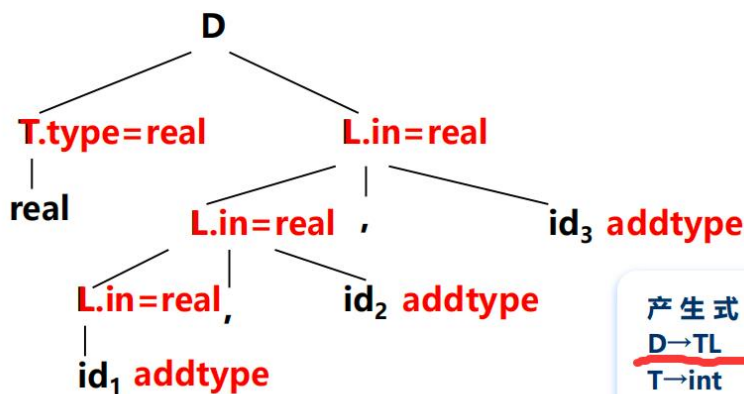
- 仅仅使用综合属性的属性文法称**S - 属性文法**



继承属性

- 在语法树中，一个结点的继承属性由此结点的父结点和/或兄弟结点的某些属性确定

- 用继承属性来表示程序设计语言结构中的上下文依赖关系很方便



产生式	语义规则
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$
$T \rightarrow real$	$T.type := real$
$L \rightarrow L_1, id$	$L_1.in := L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

- **终结符**只有**综合属性**，由词法分析器提供
- **非终结符**既可有**综合属性**也可有**继承属性** **文法开始符号的所有继承属性**作为属性计算前的初始值
- 对出现在**产生式右边的继承属性**和出现在**产生式左边的综合属性**都必须提供一个计算规则。属性计算规则中只能使用相应产生式中的文法符号的属性
- 出现在**产生式左边的继承属性**和出现在**产生式右边的综合属性**不由所给的产生式的属性计算规则进行计算，它们由其它产生式的属性规则计算或者由属性计算器的参数提供
- 语义规则所描述的工作可以包括**属性计算**、**静态语义检查**、**符号表操作**、**代码生成**等等。

4. 处理方法：依赖图、树遍历、一遍扫描

■ 基于属性文法的处理过程

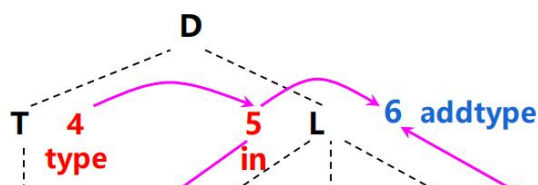
输入串 $\rightarrow$ 语法树 $\rightarrow$ 语法制导 $\rightarrow$ 语义规则计算次序

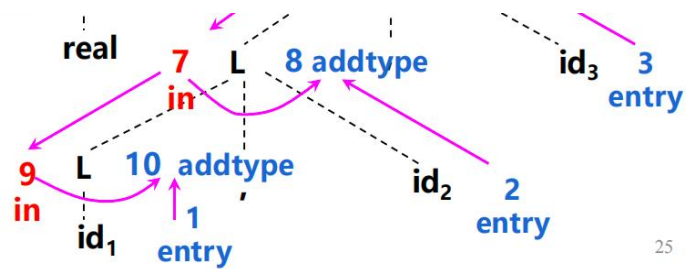
- 由源程序的语法结构所驱动的处理办法就是**语法制导翻译法**

<1>依赖图

句子 $real\ id_1,\ id_2,\ id_3$ 的带注释的语法树的依赖图

产生式	语义规则
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$
$T \rightarrow real$	$T.type := real$
$L \rightarrow L_1, id$	$L_1.in := L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$



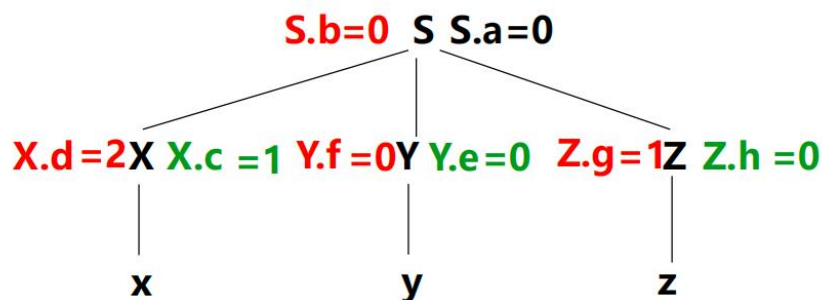


25

<2>树遍历——深度优先

假设 S.a 的初始值为0，输入串为xyz

产生式	语义规则	产生式	语义规则
$S \rightarrow XYZ$	$Z.h := S.a$	$X \rightarrow x$	$X.d := 2 * X.c$
	$X.c := Z.g$	$Y \rightarrow y$	$Y.f := Y.e * 3$
	$S.b := X.d - 2$	$Z \rightarrow z$	$Z.g := Z.h + 1$
	$Y.e := S.b$		



<3> **一遍扫描**：语法分析的时候同时计算属性值

■ 一遍扫描的处理方法是在语法分析的同时计算属性值

- 所采用的语法分析方法
- 属性的计算次序

■ S - 属性文法适合于一遍扫描的自下而上分析

5. 语法制导翻译——为每个产生式配上一个翻译子程序，并且在语法分析的时候同时执行。

- 直观上说就是为每个产生式配上一个翻译子程序（称语义动作或语义子程序），并且在**语法分析**的同时执行它。
- 语义动作一方面规定了产生式**产生的符号串**的意义，另一方面又按照这种意义规定了生成中间代码应做的**基本动作**。
- 定义：在语法分析过程中，当一个产生式**获得匹配**（自上而下分析）和**用于归约**（自下而上分析）时，此产生式对应的语义子程序进入工作，完成既定翻译任务，产生中间代码。

语法制导翻译的作用：

- 产生中间代码；
- 产生目标指令；
- 对输入串进行解释执行；

七、语义分析和中间代码生成

1. 中间代码



■ 中间语言（复杂性介于源语言和目标语言之间）的好处：

- 便于进行与机器无关的代码优化工作
- 易于移植
- 使编译程序的结构在逻辑上更为简单明确

<1>后缀式==逆波兰式

■ 一个表达式E的后缀形式可以如下定义：

1. 如果E是一个变量或常量，则E的后缀式是E自身。
2. 如果E是 $E_1 \text{ op } E_2$ 形式的表达式，其中op是任何二元操作符，则E的后缀式为 $E_1' E_2' \text{ op}$ ，其中 E_1' 和 E_2' 分别为 E_1 和 E_2 的后缀式。
3. 如果E是 (E_1) 形式的表达式，则 E_1 的后缀式就是E的后缀式。

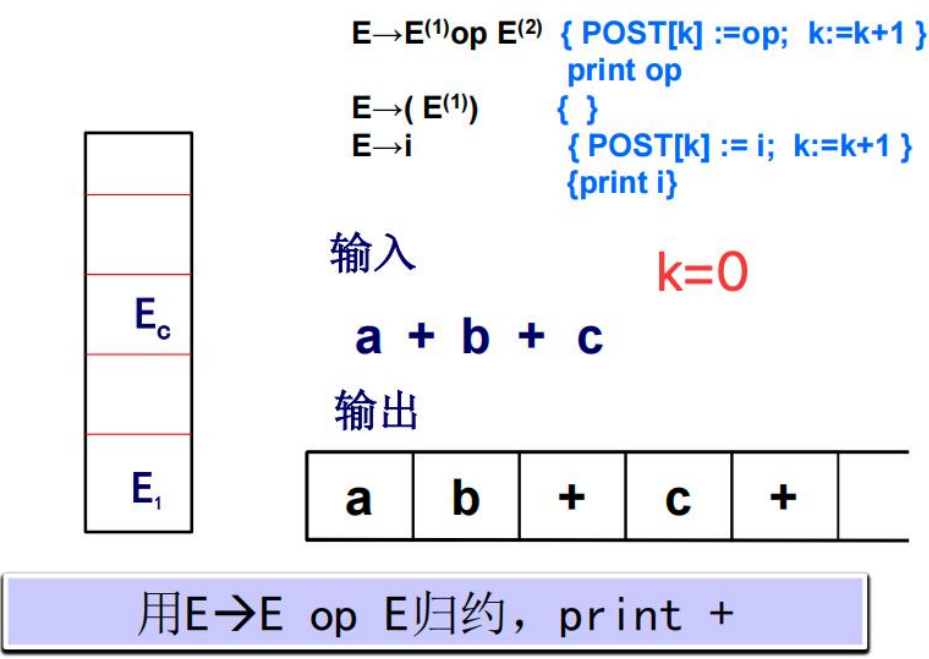
【因为逆波兰不用括号】

■ 逆波兰表示法不用括号。只要知道每个算符的目数，对于后缀式，不论从哪一端进行扫描，都能对它进行唯一分解。

■ 后缀式的计算

- 用一个栈实现。
- 一般的计算过程是：自左至右扫描后缀式，每碰到运算量就把它推进栈。每碰到k目运算符就把它作用于栈顶的k个项，并用运算结果代替这k个项。

下面这个后缀是，就是根据给定的翻译规则，每次“**归约**”动作的时候，产生相应动作！



<2>图表示（DAG、抽象语法树）

<3>三地址代码（三元式、四元式、间接三元式）

四元式：（op，arg1，arg2，result）

op	arg1	arg2	result
(0) uminus	c		T_1
(1) *	b	T_1	T_2
(2) uminus	c		T_3
(3) *	b	T_3	T_4
(4) +	T_2	T_4	T_5
(5) :=	T_5		a

- 四元式之间的联系通过临时变量实现。
 - 单目运算只用 arg1，转移语句将目标标号放入 result 域。
 - arg1，arg2，result 通常为指针，指向有关名字的符号表入口，且临时变量填入符号表。
- 三元式（op，arg1，arg2）

	op	arg1	arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

- 通过计算临时变量值的语句的位置来引用这个临时变量。

间接三元式：三元式表+间接码表（间接码表是一张指示器表，按**运算的先后次序列**出有关三元式在三元式表中的位置）

示例

语句

$X := (A + B) * C;$

$Y := D \uparrow (A + B)$

的间接三元式表示如下表所示。

间接代码	三元式表
(1)	OP ARG1 ARG2
(2)	(1) + A B
(3)	(2) * (1) C
(1)	(3) := X (2)
(4)	(4) ↑ D (1)
(5)	(5) := Y (4)

四元式、三元式和间接三元式比较

- 三元式中使用了指向三元式的指针，优化时修改较难。
- 间接三元式优化只需要更改间接码表，并节省三元式表存储空间。
- 修改四元式表也较容易，只是临时变量要填入符号表，占据一定存储空间。

2. 说明语句的翻译

- 说明部分中把**定义性出现的标识符与类型等属性**相关联，从而确定它们在计算机**内部的表示法、取值范围及可对其进行的运算**。
- 为了产生有效地可执行目标代码，对于说明部分的翻译，**不仅仅把与标识符相关联的类型等属性填入符号表中，还必须考虑到标识符所标记的对象的存储分配问题。**

<1>常量定义的翻译

- C++语言中有常量定义，以关键字const为标志，如

`const float pi=3.1416, one=1.0`

- 对每个常量定义的处理应包括下列工作：

- (1) 把等号右边的常量值登录入常量表中（若之前尚未登录）并回送常量表序号；
- (2) 然后为等号左边的标识符建立符号表新条目，在该条目中填入常量标志、相应类型及常量表序号。

> 假定常量定义中不指明类型，类型直接由常量值本身确定，且等号右边仅整数或常量标识符。

> 常量定义及相应翻译方案

CONSTDEF → const CDT

CDT → CDT; CD

CDT → CD

CD → id=num

```
{ num.ord:=lookCT(num.lexval)
  id.ord:=num.ord; id.type:=integer;
  id.kind:=CONSTANT
  add(id.entry, id.kind, id.type, id.ord)}
CD → id=id1
{id.kind:=CONSTANT; id.type:=id1.type;
 id.ord:=id1.ord
 add(id.entry, id.kind, id.type, id.ord)}
```

lookCT(c)将在常量表中查找常量c，若查不到，则将该常量值登录入常量表，最终回送常量c值在常量表中的序号

过程add是对过程addtype的扩充，它把种类、类型与序号三者填入符号表相应条目中

<2>变量说明的翻译

3. 赋值语句的翻译——会生成中间代码 emit
4. 布尔表达式的翻译——会生成中间代码 emit

■ **布尔表达式**：用**布尔运算符**把**布尔量**、**关系表达式**联结起来的式子。

- 布尔运算符：and, or, not ;
- 关系运算符 <, ≤, =, ≠, >, ≥

■ **布尔表达式的两个基本作用**：

- 用于逻辑演算，计算逻辑值；
- 用于控制语句的条件式。

■ **产生布尔表达式的文法**：

- $E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \neg E \mid (E) \mid \text{id rop id} \mid \text{id}$

■ **运算符优先级**：布尔运算由高到低：not and or，同级左结合

关系运算符同级，且高于布尔运算符

<1>数值表示法

■ **a < b 的关系表达式可等价地写成**

if a < b then 1 else 0，翻译成

```

100:    if a < b goto 103
101:    T:=0
102:    goto 104
103:    T:=1
104:

```

- 过程emit将三地址代码送到输出文件中
- nextstat: 给出输出序列中下一条三地址语句的地址索引
- 每产生一条三地址语句后，过程emit便把nextstat加1

布尔表达式 a < b or c < d and e < f 的翻译结果

```

100:    if a < b goto 103
101:    T1:=0
102:    goto 104
103:    T1:=1
104:    if c < d goto 107
105:    T2:=0
106:    goto 108
107:    T2:=1
108:    if e < f goto 111
109:    T3:=0
110:    goto 112
111:    T3:=1
112:    T4:=T2 and T3
113:    T5:=T1 or T4

```

```

E → id1 relop id2
{ E.place := newtemp;
  emit( 'if' id1.place relop.op
        id2.place 'goto' nextstat+3);
  emit(E.place ':=' '0' );
  emit( 'goto' nextstat+2);
  emit(E.place ':=' '1' ) }

E → id
{ E.place := id.place }

E → E1 or E2
{ E.place := newtemp;
  emit(E.place ':=' E1.place 'or' E2.place) }

E → E1 and E2
{ E.place := newtemp;
  emit(E.place ':=' E1.place 'and' E2.place) }

```

5. 两遍扫描条件控制及其语句翻译

“两遍”：

第一遍——为给定的输入串构造一颗语法树；

第二遍——对语法树进行深度优先遍历，进行语义规则中规定的翻译。

■ 采用优化措施

- 把A or B解释成 if A then true else B
- 把A and B解释成 if A then B else false
- 把¬ A解释成 if A then false else true

6. 一遍扫描条件控制及其语句翻译【考点】——四元式

“一遍”：没有语法树 + 等待“回填”

■ 采用四元式形式

- 把四元式存入一个数组中，数组下标就代表四元式的标号

■ 约定

四元式(jnz, a, -, p) 表示 if a goto p

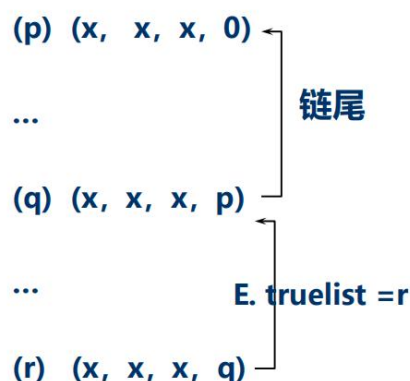
四元式(jrop, x, y, p) 表示 if x rop y goto p

四元式(j, -, -, p) 表示 goto p

■ 为非终结符E赋予两个综合属性

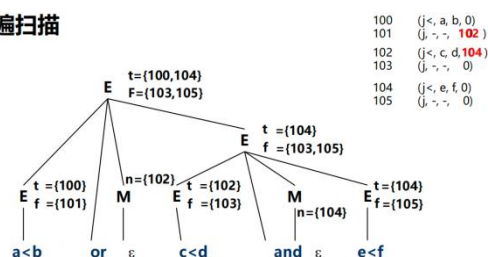
E.truelist和E.falselist。它们分别记录布尔表达式E所对应的四元式中需回填“真”、“假”出口的四元式的标号所构成的链表

- 例如：假定E的四元式中需要回填“真”出口的p, q, r三个四元式，则E.truelist为下列链：



jnz=jump not zero, 非0的时候跳转到xx

一遍扫描



7. 过程调用的处理——传递参数+过程

六七章复习完自测：

- **基本概念：语法制导翻译、逆波兰表达式**
- **能根据语义动作执行规约和相应的语义动作**
- **中间代码生成（语义动作会给，但要读懂）**
 - 布尔表达式中间代码生成，**真假出口的标定**
 - 控制语句的中间代码生成，**回填**
 - 画出来树
- **务必注意细节！！**

八、符号表【上课没讲，不作为考察重点】

<1>符号表的组织与作用

- 两个作用：收集信息、提供使用
- 符号表操作：填入、查找、访问、更新、删除
- 名字栏和信息栏
- 两种组织方式：直接方式+间接方式
- 符号表的存储：连续存储+子表存放

<2>名字的作用范围

<3>符号表的内容

- 信息栏：每个名字有关的性质——类型、种属、大小、相对数

九、运行时存储空间组织

1. 目标程序运行时的活动

- 过程：过程名+过程体
- 过程调用：过程名出现在可执行语句中
- 过程的活动：过程的一次执行
- 活动的生存期：执行过程体第一步操作到最后一步操作之间的操作序列，包括该过程调用其他过程花费的时间。

过程的活动

- 每次控制从过程P进入过程Q后，如果没有错误，最后都返回到过程P。
 - 对于非递归的两个过程，如果a和b都是过程的活动，那么它们的生存期或者是不重叠的，或者是嵌套的。
- 如果一个过程是递归的，那么在某一时刻可能有这个过程的几个活动活跃着。
- 编译程序组织存储空间时，要考虑激活一个过程的新活动时或从过程的活动返回时，对局部名的处理，及是否允许静态或动态存储分配等。
- 过程（函数）是结构化程序设计的主要手段，同时也是节省程序代码和扩充语言能力的主要途径。
- 过程定义：

```
procedure add(x,y:integer; var z:integer)
begin
    z: =x+y;
end;
```

- 过程调用

```
add(a,b,c);
```

- 调用和被调用过程之间的信息是通过参数（或全局量）来传递的。称之为形式参数与实在参数。——掌握四种参数传递的方式

<1>传地址

- 过程体中对形式参数的引用或赋值都会形成对形式单元的间接访问——实时改变。

- 把实在参数的地址传递给相应的形式参数，形式参数在过程段有相应的形式单元存放相应的实在参数的地址。

示例**对如下程序**

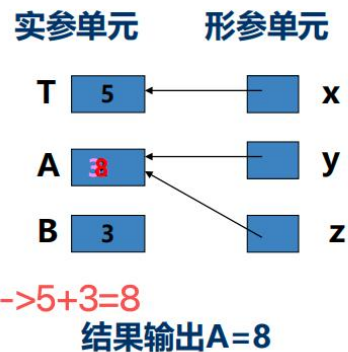
```

procedure P(x,y,z)
begin
    y:=y+1;
    z:=z+x;
end
program M(input, output)
begin
    A:=2;B:=3;
    P(A+B,A,A);
    print A;
end

```

传地址方式下的程序输出结果。

传地址方式就是实时以地址所指的值为准



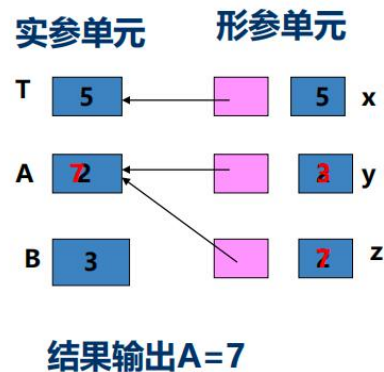
<2>得结果——传地址的一种变形，但不等价

- 每个形参对应两个形式单元，一个放实参的地址，一个放实参的值；
- 过程体中的处理仅仅只对形式单元的值进行直接访问，过程完成返回前才把这第二个形式单元的值放到第一个单元地址所指的实参中。

```

procedure P(x,y,z)
begin
    y:=y+1;
    z:=z+x;
end
program M(input, output)
begin
    A:=2;B:=3;
    P(A+B,A,A);
    print A;
end

```

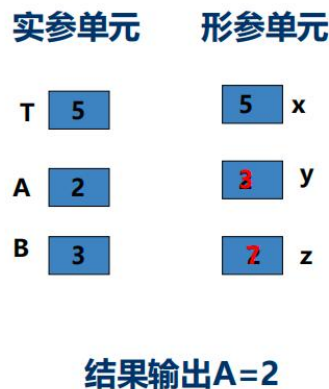
得结果方式下的程序输出结果。

<3>传值——只是把实参的值传递给对应的形式参数，不会影响实参原本的值

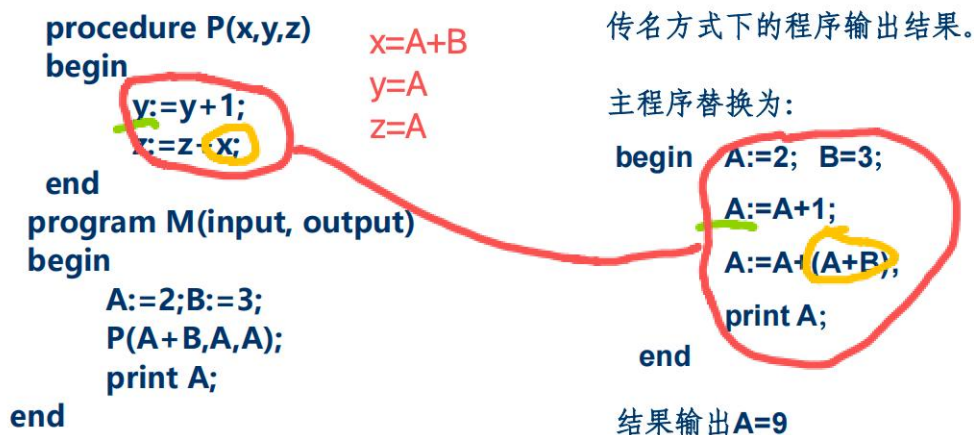
```

procedure P(x,y,z)
begin
    y:=y+1;
    z:=z+x;
end
program M(input,
output)
begin
    A:=2;B:=3;
    P(A+B,A,A);
    print A;
end

```

传值方式下的程序输出结果。

<4>传名——直接把实参的“原始表达式”原封不动地抄进函数体里，等用的时候再算，而且每次用到都重新算一次。理解为文本替换（像宏展开），而不是“数值传递”。



17

例题

```
...
procedure P(w,x,y,z);
begin
  y := y*w;
  z := z+x;
end
begin
  a := 5;
  b := 3;
  P(a+b,a-b,a,a);
  write(a);
end
```

1.传值: (形参) a原来是多少就是多少;
2.传地址: 实时改变--a=a*8=40; a=a+2=42;
3.得结果: 过程结束前才改变--y=y*8=40; z=5+2=7; 7写入;
4.传名: a=a* (a+b) =40; a=a+ (a-b) =40+ (40-3) =77;

传值:	5
传地址:	42
得结果:	7
传名:	77

19

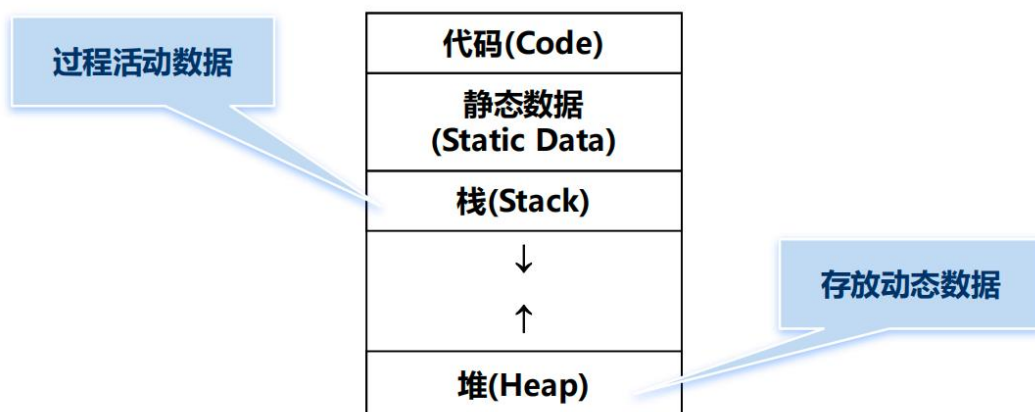
作用域: 一个名字能被使用的区域范围称作这个名字的作用域; 允许同一个标识符在不同的过程中代表不同的名字。

2. 运行时存储器的划分

■ 一个目标程序运行所需的存储空间包括:

- 存放目标代码的空间
- 存放数据项目的空间
- 存放程序运行的控制或连接数据所需单元 (控制栈)

■ 一个程序在运行时刻的内存划分:

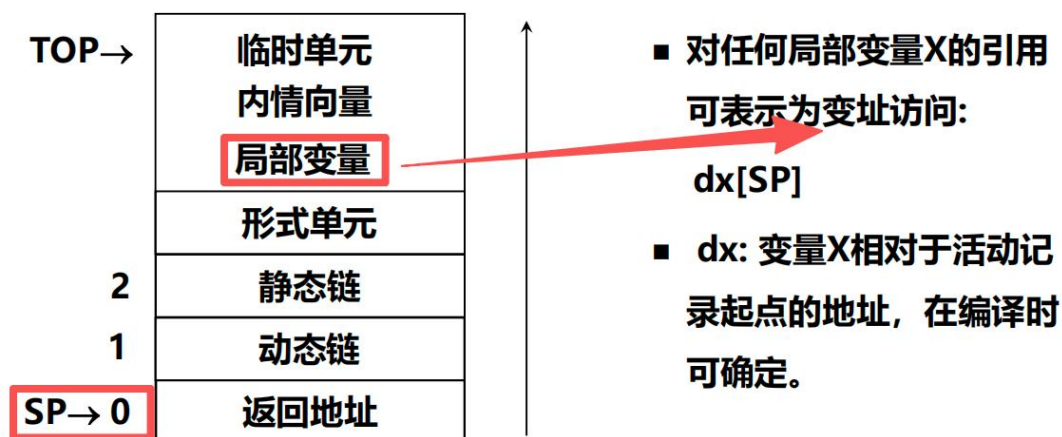


· 活动记录:

- **活动记录**: 为了管理一个过程活动所需的信息, 使用一个连续的存储块存放这些信息, 这个连续存储块称为**活动记录**。
- 运行时, 每当进入一个过程就有一个相应的活动记录累筑于栈顶。此记录含有**连接数据**、**形式单元**、**局部变量**、**局部数组**的内情向量和临时工作单元等。

以下是要背下来的:





21

3. 静态存储分配

- **静态分配策略(FORTRAN):** 如果在编译时能确定数据空间的大小, 则可采用静态分配方法。在编译时刻为每个数据对象分配固定的存储单元, 且在运行时始终保持不变。
- **动态分配策略(PASCAL):** 如果在编译时不能确定运行时数据空间的大小, 则必须采用动态分配方法。允许递归过程和动态申请释放内存。
 - 栈式动态分配
 - 堆式动态分配



■ 考虑子程序段:

SUBROUTINE SWAP(A,B)

T=A

A=B

B=T

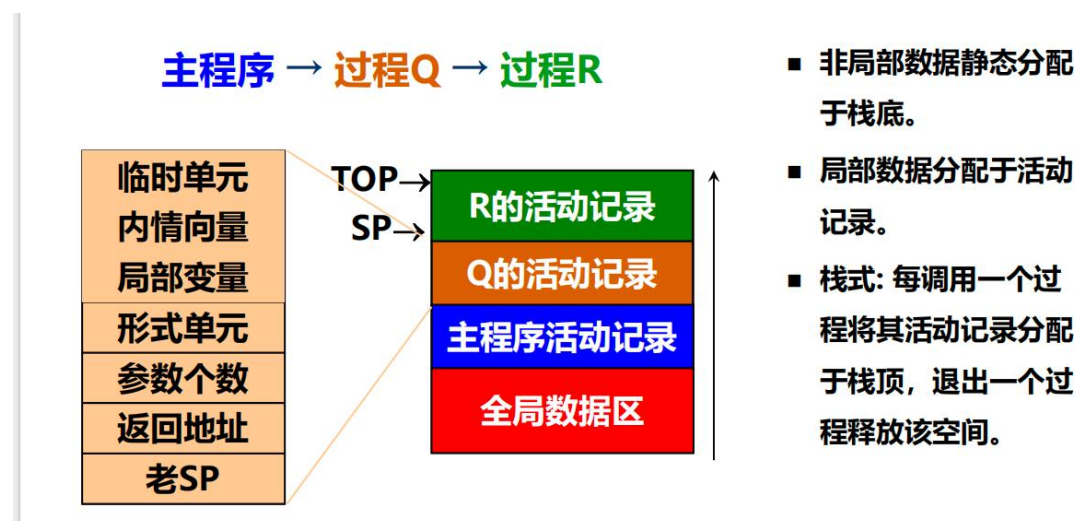
RETURN

END

名字 NAME	性质 ATTRIBUTE	地址	
		DA	ADDR
SWAP	子程序, 二目		
A	哑, 实变量	k	a
B	哑, 实变量	k	a+2
T	实变量	k	a+4



4. 简单的栈式存储分配



42

5. 嵌套过程语言的栈式实现【重点】

非局部名字的访问的实现

- 主程序的层次为**0**; 在*i*层中定义的过程, 其层次为*i*+1;
(层数, 偏移量)
- 过程运行时, 必须知道其所有外层过程的当前活动记录的起始地址。
 - 嵌套层次显示表display
 - 静态链和活动记录

嵌套层次显示表display

- 当进入一个过程后, 在建立其活动记录区的同时建立一张嵌套层次显示表display, 把display表作为活动记录的一部分。
- 令过程R的外层为Q, Q的外层为主程序P, 则过程R运行时的DISPLAY表内容为:

2	R 的现行活动记录的地址(SP 的现值)
1	Q 的最新活动记录的地址
0	P 的活动记录的地址

55

- 问题：当过程P1调用过程P2而进入P2后，P2应如何建立起自己的display表？

从P1的display表中自底而上地取过 $l/2$ 个单元（ $l/2$ 为P2的层数）再添上进入P2后新建立的SP值就构成了P2的display表。



5

- 问题：当过程P1调用过程P2而进入P2后，P2应如何建立起自己的display表？

从P1的display表中自底而上地取过 $l/2$ 个单元（ $l/2$ 为P2的层数）再添上进入P2后新建立的SP值就构成了P2的display表。



- 问题：当过程P1调用过程P2而进入P2后，P2应如何建立起自己的display表？

从P1的display表中自底而上地取过 $l/2$ 个单元（ $l/2$ 为P2的层数）再添上进入P2后新建立的SP值就构成了P2的display表。



14

下面这个要背住：



- diaplay表在活动记录中的相对地址d在编译时能完全确定。
- 假定在现行过程中引用了某层过程(令其层次为k)的X变量，那么，可用下面两条指令获得X的地址:

LD R₁ (d+k)[SP]

LD R₂ dx[R₁]

```

program P;
var a, x : integer;
procedure Q(b: integer);
  var i: integer;
  procedure R(u: integer; var v: integer);
  var c, d: integer;
  begin
    if u=1 then R(u+1, v)
    .....
    v:=(a+c)*(b-d);
    .....
  end {R}
begin
  .....
  R(1,x);
  .....
end {Q}
  
```

```

procedure S;
var c, i: integer;
begin
  a:=1;
  Q(c);
  .....
end {S}

begin
  a:=0;
  S;
  .....
end. {P}
  
```

主程序 P → 过程 S → 过程 Q
→ 过程 R → 过程 R

62

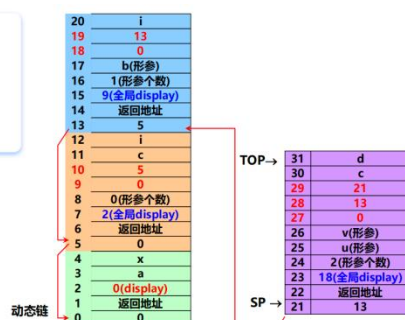
主程序 P →
过程 S



主程序 P →
过程 S →
过程 Q



主程序 P →
过程 S →
过程 Q →
过程 R



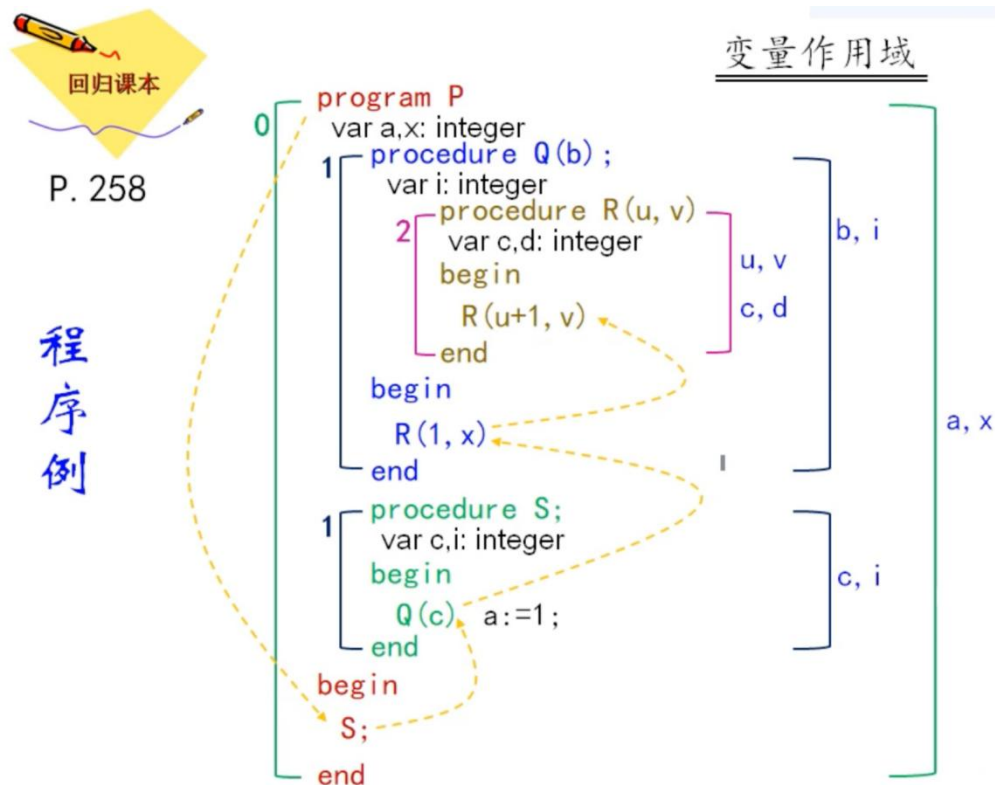
主程序 P →
过程 S →
过程 Q →
过程 R



第九章一些内容的澄清:

“嵌套过程语言”指的是: 不仅函数调用时可以嵌套调用, 就连函数定义, 都可以**嵌套定义**!!!!

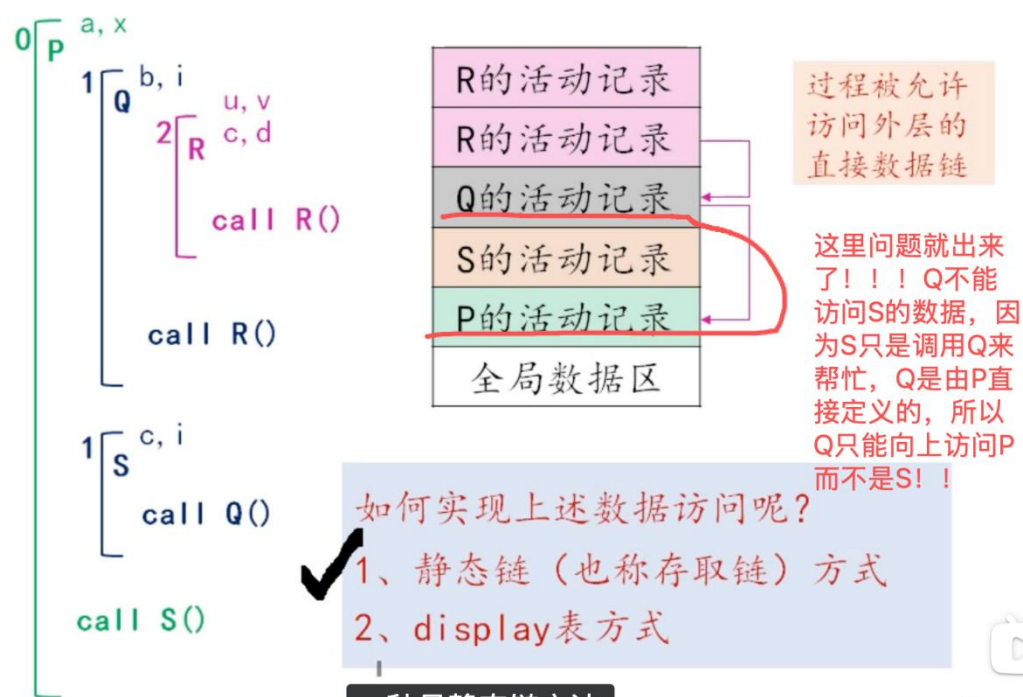
(这里也是 display 表构建时的**大坑**!!) 典型的语言是 pascal



区分这里的定义层次和调用层次: Q 和 S 是同一定义层次的!!

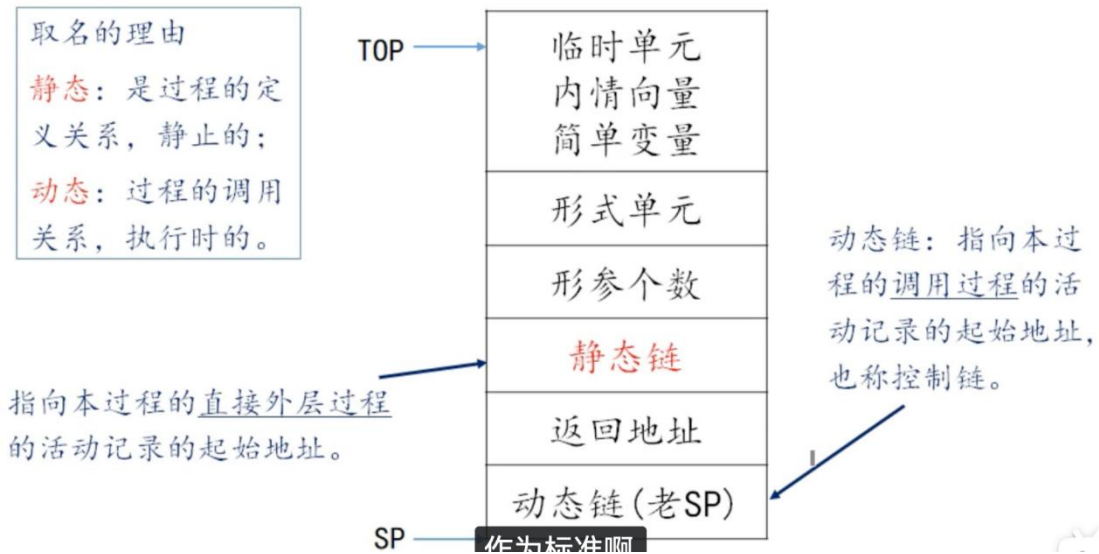
活动记录按照调用关系 (层次) 来建立, 但是**数据访问**按照**定义**关系 (层次) 确定!!

程序调用大观



可以这么理解，因为静态链和 display 解决的是同一个问题嘛，所以起作用的范围应该也是一样的。而对于静态链来讲——“静态”means 编译的时候就确定=》只能是根据定义关系呀！

三、解决方案（1）：使用静态链



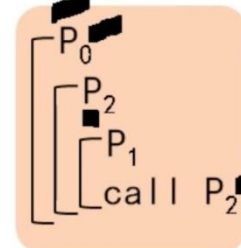
静态链的建立

- **情况1**：第N层过程(P_1)调用第N+1层过程(P_2)
 - P_2 的静态链为调用过程 P_1 的最新活动记录的起始地址。
- **情况2**：第N层过程(P_1)调用第N层过程(P_2)
 - P_2 的静态链为调用过程 P_1 的静态链的值；
- **情况3**：第N层过程(P_1)调用第N-x层过程(P_2)
 - P_2 的静态链为沿着调用过程 P_1 的静态链向前走x步到达的活动记录的静态链的值。

总之静态链必须指向定义关系中的直接“父亲”！！而跟调用过程无关！！



好那么这个时候



感觉对于这章的理解还不够深刻，复习完还有时间的话可以再详细看看网课。

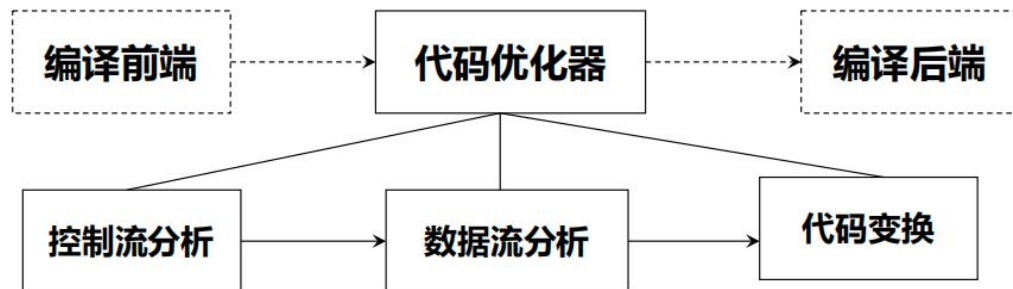
八九章复习完自测：

- **基本概念：符号表作用（贯穿整个编译过程，每个阶段的作用）**
- **熟练掌握参数传递的4种方式**
- **运行存储策略中动态链、静态链和DISPLAY表分别的作用**
- **熟练掌握嵌套过程运行栈的构造**
- **运行栈的内容**
- **DISPLAY表概念和构建方法**

十、优化

1. 优化

- 优化：对程序进行各种等价变换，使得从变换后的程序出发，能生成更有效的目标代码。



- **等价原则**：经过优化后不应改变程序运行的结果；
- **有效原则**：使优化后所产生的目标代码运行时间较短，占用的存储空间较小；
- **合算原则**：应尽可能以较低的代价取得较好的优化效果。

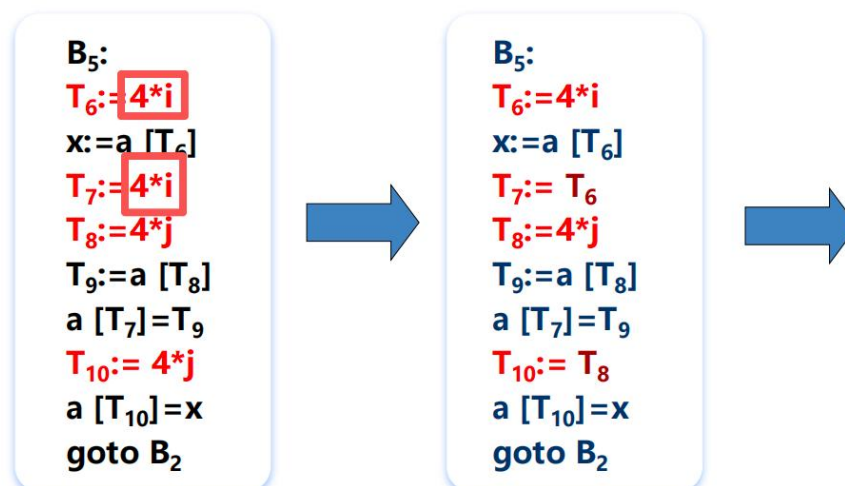
■ 优化的三个不同级别：

- 局部优化
- 循环优化
- 全局优化

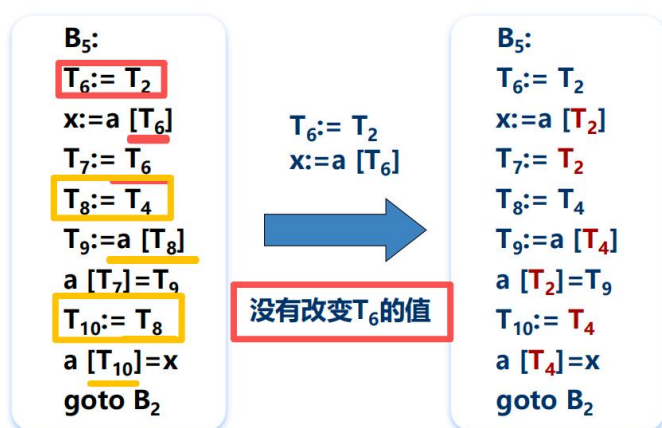
■ 优化的种类：

- 删除多余运算(或称删除公用子表达式)
- 代码外提
- 强度削弱
- 变换循环控制条件
- 合并已知量
- 复写传播
- 删除无用赋值

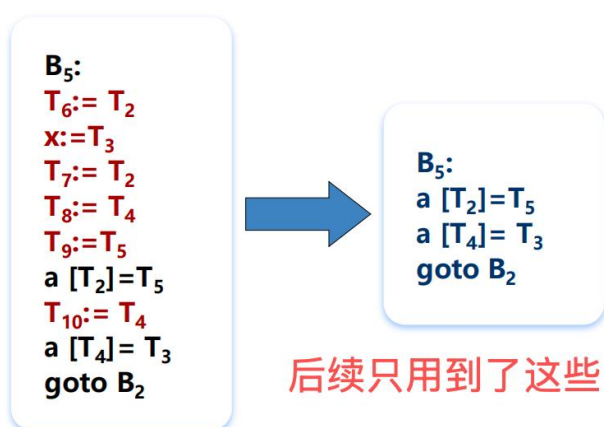
- 删除公用子表达式



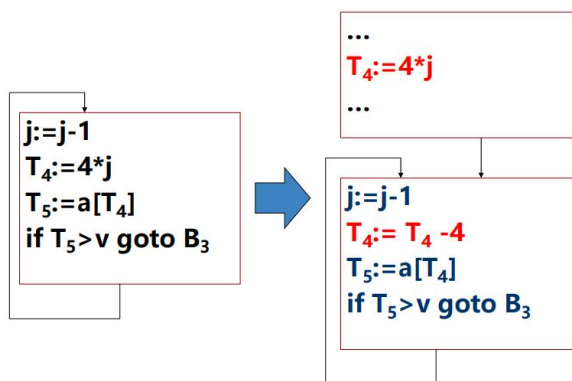
· 复写传播



· 删除无用代码



· 强度削弱 (针对循环优化) ——乘法变加法后会更快!



· 删除归纳变量

- B_2 中的 i 与 T_2 的值保持着线性关系
- B_3 中的 j 与 T_4 的值保持着线性关系
- 此种变量称之为归纳变量
- 对该类变量也可进行优化

2. 局部优化 (块内优化)

· 划分基本块

- **基本块**: 指程序中一顺序执行语句序列, 其中只有一个入口和一个出口。**入口**就是其中第一个语句, **出口**就是其中最后一个语句。
- 如果一条三地址语句为 $x:=y+z$, 则称对 x **定值并引用** y 和 z 。
- 在一个基本块中的一个名字, 所谓在程序中的某个给定点是**活跃的**, 是指如果在程序中(包括在本基本块或在其它基本块中)它的值在该点以后被引用。

■ 划分四元式程序为基本块的算法:

1. 求出四元式程序中各个基本块的入口语句:

- 1) **程序第一个语句**, 或
- 2) **能由条件转移语句或无条件转移语句转移到的语句**, 或
- 3) **紧跟在条件转移语句后面的语句**。

2. 对以上求出的每个入口语句, 确定其所属的基本块。它是由该入口语句到**下一入口语句**(不包括该入口语句)或到**一转移语句**(包括该转移语句)或**一停语句**(包括该停语句)之间的语句序列组成的。



3. 凡未被纳入某一基本块中的语句, 可以从程序中删除。

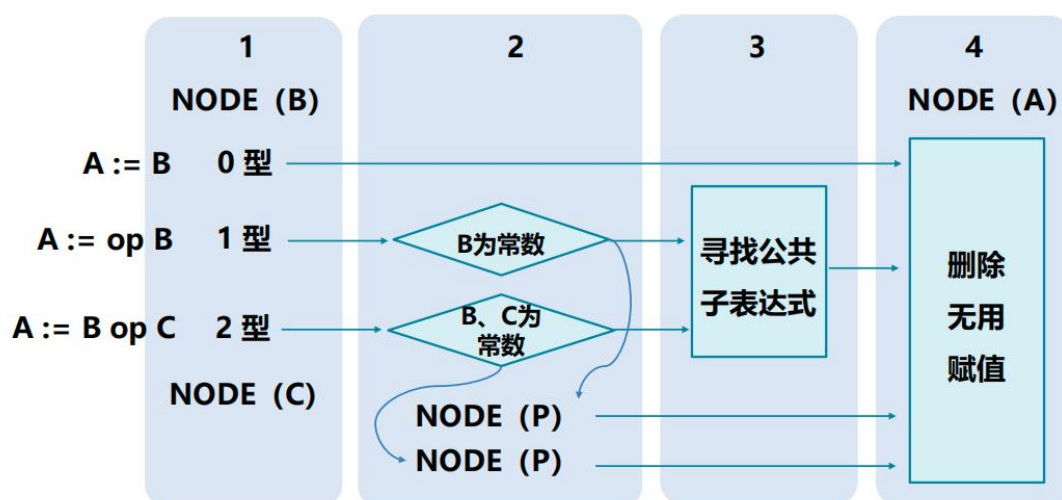
36

(1)	read X
(2)	read Y
(3)	R:=X mod Y
(4)	if R=0 goto (8)
(5)	X:=Y
(6)	Y:=R
(7)	goto (3)
(8)	write Y
(9)	halt

2. 对以上求出的每个入口语句, 确定其所属的基本块。它是由该入口语句到**下一入口语句**(不包括该入口语句)、或到**一转移语句**(包括该转移语句)、或**一停语句**(包括该停语句)之间的语句序列组成的。

· 块内优化措施：删除公共子表达式、删除无用赋值、合并已知量、临时变量改名、交换语句位置、代数变换

【其实块内优化也不用记上面那么多，主要就是 DAG 图优化就行，DAG 图的原理步骤看着复杂，其实实操起来很简单】



基本块的DAG构造算法

1. 准备操作数的结点

- 如果NODE(B)无定义，则构造一标记为B的叶结点并定义NODE(B)为这个结点；
- 如果当前四元式是0型，则记NODE(B)的值为n，转4。
- 如果当前四元式是1型，则转2(1)
- 如果当前四元式是2型，则(i)如果NODE(C)无定义，则构造一标记为C的叶结点并定义NODE(C)为这个结点；(ii)转2(2)。

2. 合并已知量

- (1) 如果NODE(B)是标记为常数的叶结点，则转2(3)；否则，转3(1)。
- (2) 如果NODE(B)和NODE(C)都是标记为常数的叶结点，则转2(4)；否则，转3(2)。
- (3) 执行op B (即合并已知量)。令得到的新常数为P。如果NODE(B)是处理当前四元式时新构造出来的结点，则删除它。如果NODE(P)无定义，则构造一用P作标记的叶结点n。置NODE(P)=n，转4。
- (4) 执行B op C (即合并已知量)。令得到的新常数为P。如果NODE(B)或NODE(C)是处理当前四元式时新构造出来的结点，则删除它。如果NODE(P)无定义，则构造一用P作标记的叶结点n。置NODE(P)=n，转4。

3. 寻找公共子表达式

(1) 检查DAG中是否已有一结点，其唯一后继为NODE(B)且标记为op(即公共子表达式)。如果没有，则构造该结点n，否则，把已有的结点作为它的结点并设该结点为n。转4。

(2) 检查DAG中是否已有一结点，其左后继为NODE(B)，右后继为NODE(C)，且标记为op(即公共子表达式)。如果没有，则构造该结点n，否则，把已有的结点作为它的结点并设该结点为n。转4。

4. 删除无用赋值

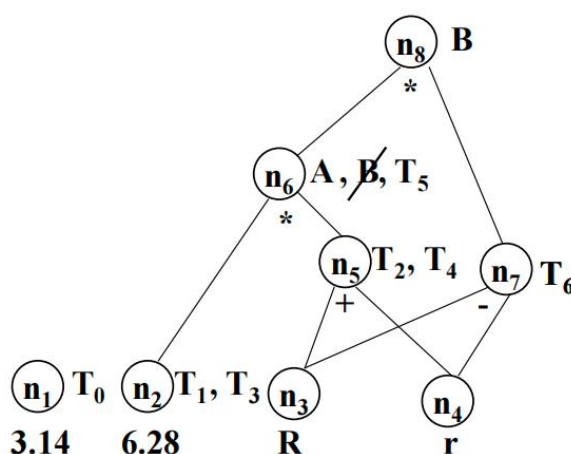
如果NODE(A)无定义，则把A附加在结点n上并令NODE(A)=n；否则，先把A从NODE(A)结点上的附加标识符集中删除

(注意，如果NODE(A)是叶结点，则其A标记不删除)。

把A附加到新结点n上并置NODE(A)=n。转处理下一四元式。

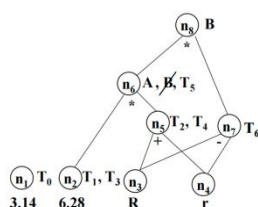
示例：试构造以下基本块G的DAG

- (1) $T_0 := 3.14$
- (2) $T_1 := 2 * T_0$
- (3) $T_2 := R + r$
- (4) $A := T_1 * T_2$
- (5) $B := A$
- (6) $T_3 := 2 * T_0$
- (7) $T_4 := R + r$
- (8) $T_5 := T_3 * T_4$
- (9) $T_6 := R - r$
- (10) $B := T_5 * T_6$



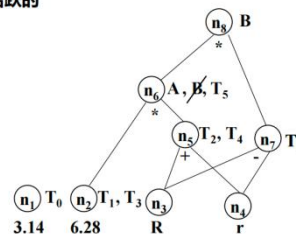
优化后的四元式

- (1) $T_0 := 3.14$
- (2) $T_1 := 6.28$
- (3) $T_3 := 6.28$
- (4) $T_2 := R + r$
- (5) $T_4 := T_2$
- (6) $A := 6.28 * T_2$
- (7) $T_5 := A$
- (8) $T_6 := R - r$
- (9) $B := A * T_6$



若只有A和B是出基本块之后活跃的

- (1) $T_2 := R + r$
- (2) $A := 6.28 * T_2$
- (3) $T_6 := R - r$
- (4) $B := A * T_6$
- (1) $S_1 := R + r$
- (2) $A := 6.28 * S_1$
- (3) $S_2 := R - r$
- (4) $B := A * S_2$



- 在基本块外被定值并在基本块内被引用的所有标识符，就是作为叶子结点上标记的那些标识符。
- 在基本块内被定值并且该值在基本块后面可以被引用的所有标识符，就是DAG各结点上的那些附加标识符。

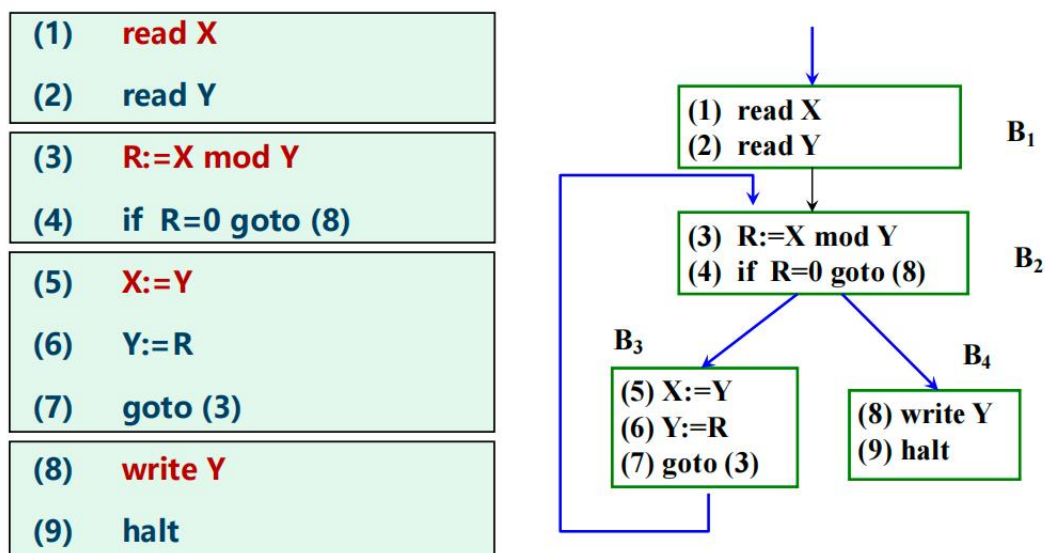
3. 循环优化

流图

- 每个流图以基本块为**结点**。如果一个结点的基本块的入口语句是程序的第一条语句，则称此结点为**首结点**。如果在某个执行顺序中，基本块 B_2 紧接在基本块 B_1 之后执行，则从 B_1 到 B_2 有一条有向边。即，如果
 - 有一个条件或无条件转移语句从 B_1 的最后一条语句转移到 B_2 的第一条语句；或者
 - 在程序的序列中， B_2 紧接在 B_1 的后面，并且 B_1 的最后一条语句不是一个无条件转移语句。

我们就说 B_1 是 B_2 的**前驱**， B_2 是 B_1 的**后继**。

60



· 循环优化手段：代码外提、强度削弱、删除归纳变量（变换循环控制条件）、循环展开、循环合并。

代码外提：把循环不变运算提到循环体外

- **循环不变运算**: 对四元式 $A := B \text{ op } C$, 若 B 和 C 是常数, 或者到达它们的 B 和 C 的定值点都在循环外。
- 变量 A 在某点 d 的 **定值到达** 另一点 u (或称变量 A 的定值点 d 到达另一点 u), 是指流图中从 d 有一通路到达 u 且该通路上没有 A 的其它定值。

$d : A := B \text{ op } C$

...

$u : D := A \text{ op } E$

- 把循环不变运算提到循环体外。

代码外提算法

1. 求出 L 的所有不变运算
2. 对每个不变运算 $s: A := B \text{ op } C$ 或 $A := \text{op } B$ 或 $A := B$ 检查是否满足条件(1)或(2)

(1)

- (i) s 所在的结点是 L 所有出口结点的必经结点;
- (ii) A 在 L 中其他地方未再定值;
- (iii) L 中所有 A 的引用点只有 s 中的 A 的定值才能到达。

代码外提算法

(2) A 在离开 L 后不再是活跃的, 并且条件(1)的(ii)和(iii)成立。所谓 **A 在离开 L 后不是活跃的** 是指, A 在 L 的任何出口结点的后继结点入口处不是活跃的。

3. 按步骤1所找出的不变运算的次序, 依次把符合2的条件(1)或(2)的不变运算 s 外提到 L 的前置结点中。但是, 如果 s 的运算对象(B 或 C)是在 L 中定值的, 那么, 只有当这些定值四元式都已外提到前置结点中时, 才能把 s 也外提到前置结点中。

强度削弱: 把程序中执行时间较长的运算转换为执行时间较短的运算。比如把循环中的乘法运算用递归加法运算替换

总结

- 强度削弱主要是对与归纳变量有线性关系的变量赋值进行;
- 经过强度削弱后, 循环中可能出现一些新的无用赋值;
- 对于削弱下标变量地址计算的强度非常有效。

删除归纳变量:

- 如果循环中对变量I只有唯一的形如 $I := I \pm C$ 的赋值, 且其中C为循环不变量, 则称I为循环中的基本归纳变量。
- 如果I是循环中一基本归纳变量, J在循环中的定值总是可化归为I的同一线性函数, 也即 $J = C_1 * I \pm C_2$, 其中 C_1 和 C_2 都是循环不变量, 则称J是归纳变量, 并称它与I同族。
- 一个基本归纳变量也是一归纳变量。

强度削弱和删除归纳变量的统一算法

1. 利用循环不变运算信息, 找出循环中所有基本归纳变量。
2. 找出所有其它归纳变量A, 并找出A与已知基本归纳变量X的同族线性函数关系 $F_A(X)$ 。
3. 对2中找出的每一归纳变量A, 进行强度削弱。
4. 删除对归纳变量的无用赋值。
5. 删除基本归纳变量。如果基本归纳变量B在循环出口之后不是活跃的, 并且在循环中, 除在其自身的递归赋值中被引用外, 只在形如

if B rop Y goto L

中被引用, 则可选取一与B同族的归纳变量M来替换B进行条件控制。最后删除循环中对B的递归赋值的代码。

这一章看完PPT感觉还是学的不扎实——这里再结合mooc上国防科大的讲解做一个小的总结

优化的基本概念

- **优化**：对程序进行各种等价变换，使得从变换后的程序出发，能生成更有效的目标代码。
 - **等价**：不改变程序的运行结果
 - **有效**：目标代码运行时间短，占用存储空间小



优化可以在 1. 中间代码层面，也可以在 2. 目标代码层面。

但是我们学习的方法主要针对的是中间代码。通过数据流、控制流的分析来实现代码变换。

我们学习的优化主要针对局部优化（基本块内优化）和循环优化（控制结构中）（暂不考虑全局优化）

目前针对一道优化题目，主要进行：

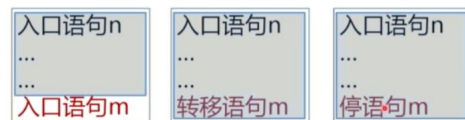
1. 删除公共子表达式
2. 复写传播
3. 删除多余赋值
4. 强度削弱与删除归纳变量。

知识点：1. 划分基本块（入口语句、出口语句、定值、引用、活跃）

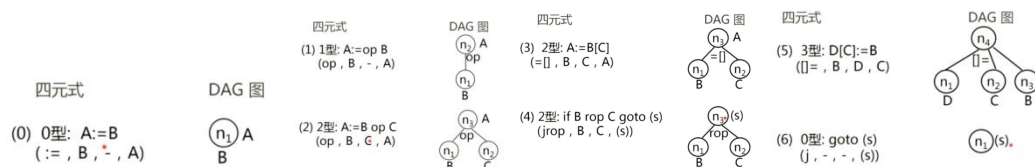
1. 找出中间语言(三地址语句)程序中各个基本块的入口语句：

- 1) 程序第一个语句，或
- 2) 能由条件转移语句或无条件转移语句转移到的语句或
- 3) 紧跟在条件转移语句后面的语句

2. 对以上求出的每个入口语句，确定其所属的基本块。它是由该入口语句到下一入口语句(不包括该入口语句)、或到一转移语句(包括该转移语句)、或一停语句(包括该停语句)之间的语句序列组成的。



2. 基本块的 DAG 表示及优化



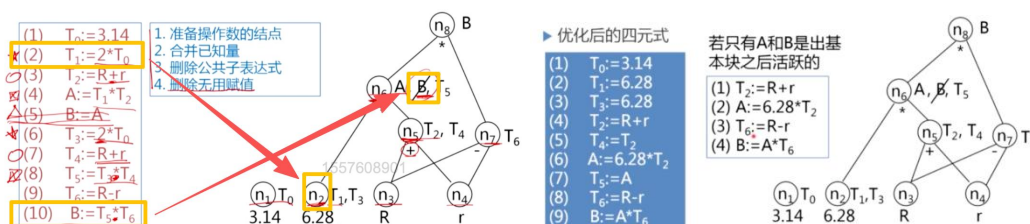
基本块内优化算法：4 步

<1>准备操作数结点

<2>合并已知量【如果操作数是常量，则在编译时合并，不再生成计算代码】

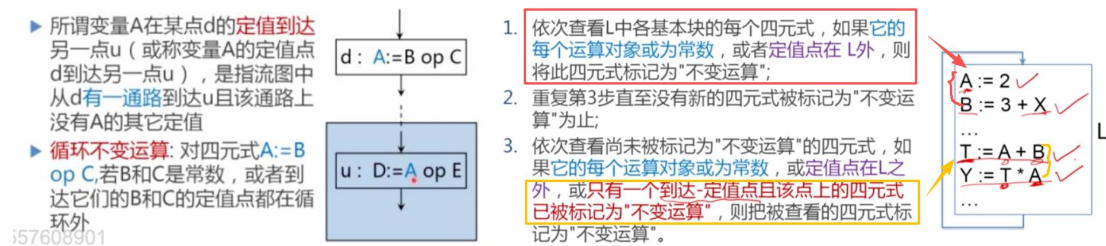
<3>删除公共子表达式【运算代码之前算过】

<4>删除无用赋值【生成 DAG 过程中找到无用赋值并删掉】



循环优化的代码优化措施：**代码外提、强度消弱、删除归纳变量（变换循环控制条件）、循环展开、循环合并【只考虑前三个】**

<1>代码外提：定值到达、循环不变运算、查找循环不变运算



并非只要是循环不变运算就可以进行代码外提——代码外提的条件（3）【同时满足】：

- 不变运算出了循环后是**不活跃**的/所在结点是循环L**所有**出口结点的必经结点；
- 不变运算的定值点在循环L中其他地方**未再定值**；
- 循环L中所有A的**引用点**只有S中的A的**定值**才能**到达**；

1. 求出L的所有不变运算

2. 对每个**不变运算** $s: A:=B \text{ op } C$ 或 $A:=\text{op } B$ 或 $A:=B$ 检查是否满足条件(1)或(2)

条件(1)
(i) S所在的结点是L所有出口结点的必经结点；
(ii) A在L中其他地方未再定值；
(iii) L中所有A的引用点只有S中的A的定值才能到达。

条件(2)
(i) A在离开L后不再是活跃的；
(ii) A在L中其他地方未再定值；
(iii) L中所有A的引用点只有S中的A的定值才能到达。

注意依赖关系

3. 按步骤1所找出不变运算的**次序**，依次把符合步骤2的条件(1)或(2)的**不变运算s**外提到L的前置结点中。但是，如果s的运算对象(B或C)是在L中定值的，那么，只有当这些定值四元式都已外提到前置结点中时，才能把s也外提到前置结点中。

<2>强度消弱——执行时间较长的运算转换为执行时间较短的运算 eg 乘法变加法

强度消弱

- ▶ 强度消弱通常是针对**循环控制变量有线性关系**的变量赋值进行
- ▶ 经过强度消弱后，循环中可能出现一些新的无用赋值
- ▶ 对于消弱下标变量地址计算的强度非常有效

强度消弱可能会破掉一些依赖关系

<3>删除归纳变量——与强度消弱紧密相关

基本归纳变量、归纳变量、同族【循环不变量、加法、线性关系】

- ▶ 如果循环中对变量I只有唯一的形如 $I:=I \pm C$ 的赋值，且其中C为循环不变量，则称I为循环中的**基本归纳变量**
- ▶ 如果I是循环中一基本归纳变量，J在循环中的定值总是可化归为I的同一线性函数，也即 $J=C_1 * I \pm C_2$ ，其中 C_1 和 C_2 都是循环不变量，则称J是**归纳变量**，并称它与I同族。基本归纳变量也是一归纳变量。

删除归纳变量

- ▶ 删除归纳变量在强度消弱以后进行
- ▶ 强度消弱和删除归纳变量的统一算法框架

- 利用**循环不变运算**信息，找出循环中所有**基本归纳变量**。
- 找出所有其它归纳变量A，并找出A与已知基本归纳变量X的同族线性函数关系 $F_A(X)$ 。
- 对2中找出的每一归纳变量A，进行强度消弱。
- 删除对归纳变量的无用赋值。
- 删除基本归纳变量。如果基本归纳变量B在循环出口之后**不活跃**的，并且在循环中，除在其自身的递归赋值中被引用外，只在形如 `if B rop Y goto L` 的语句中被引用，则可选取一与B同族的归纳变量M来替换B进行条件控制。最后删除循环中对B的递归赋值的代码。

第十章复习完自测：

- **划分基本块，程序流图**
- **熟悉基本优化方法：**局部优化和循环优化分别对应哪些优化方法
- **熟练掌握局部优化：**用DAG图优化，能够写出优化后的四元式表示
- **熟练掌握循环优化：**代码外提条件、强度削弱、删除基本归纳变量

十一、 目标代码生成

1. 基本问题

■ 代码生成器

- 代码生成器的输入包括源程序的中间表示，以及符号表中的信息
 - 利用符号表信息决定数据对象运行时地址
 - 类型检查



- **代码生成**是把语义分析后或优化后的中间代码变换成目标代码。

■ 代码生成着重考虑的问题

- 如何使生成的目标代码较短；**短**
- 如何充分利用计算机的寄存器，减少目标代码中访问存贮单元的次数。**减少访存**
- 如何充分利用计算机的指令系统的特点。**与机器有关**

■ 目标代码一般有以下三种形式：

- **能够立即执行的机器语言代码**，所有地址已经定位；
- **待装配的机器语言模块**。执行时，由连接装配程序把它们和某些运行程序连接起来，转换成能执行的机器语言代码；
- **汇编语言代码**。尚须经过汇编程序汇编，转换成可执行的机器语言代码。

<1>指令选择：一致性和完整性；指令速度和机器用语；

<2>寄存器分配：最优寄存器指派是 NPC 问题（NP 完全问题）

<3>计算顺序选择

2. 目标机器模型

■ 考虑一个抽象的计算机模型

- 具有多个通用寄存器，他们既可以作为累加器，也可以作为变址器。
- 运算必须在某个寄存器中进行。
- 含有四种类型的指令形式

类 型	指令形式	意义(设 op 是二目运算符)
直接地址型	op <u>R_i</u> , M	$(R_i) \text{ op } (M) \Rightarrow R_i$
寄存器型	op <u>R_i</u> , R _j	$(R_i) \text{ op } (R_j) \Rightarrow R_i$
变址型	op <u>R_i</u> , c(R _j)	$(R_i) \text{ op } ((R_j)+c) \Rightarrow R_i$
间接型	op <u>R_i</u> , *M	$(R_i) \text{ op } ((M)) \Rightarrow R_i$
	op <u>R_i</u> , *R _j	$(R_i) \text{ op } ((R_j)) \Rightarrow R_i$
	op <u>R_i</u> , *c(R _j)	$(R_i) \text{ op } (((R_j)+c)) \Rightarrow R_i$

如果op是一目运行符，则“op R_i, M”的意义为：

op (M) $\Rightarrow$ R_i, 其余类型可类推。

3. 一个简单的代码生成器

- 不考虑代码的执行效率，目标代码生成是不难的，例如：

翻译为四元式：

A := (B+C)*D+E

T₁ := B+C

T₂ := T₁*D

T₃ := T₂+E

A := T₃

假设只有一个寄存器可供使用

四元式

T₁ := B+C

T₂ := T₁*D

T₃ := T₂+E

A := T₃

目标代码

```
LD R0, B
ADD R0, C
ST R0, T1
LD R0, T1
MUL R0, D
ST R0, T2
LD R0, T2
ADD R0, E
ST R0, T3
LD R0, T3
ST R0, A
```

假设T₁, T₂, T₃在本块之后不再引用：

```
LD R0, B
ADD R0, C
MUL R0, D
ADD R0, E
ST R0, A
```

- 四元式的中间代码变换成目标代码，并在一个基本块的范围内考虑如何充分利用寄存器：

➢ **尽可能留：**在生成计算某变量值的目标代码时，尽可能让该变量保留在寄存器中。

➢ **尽可能用：**后续的目标代码尽可能引用变量在寄存器中的值，而不访问内存。

➢ **及时腾空：**在离开基本块时，把存在寄存器中的现行的值放到主存中。

- 如果在一个基本块内 四元式*i*对A定值，四元式*j*要引用A值，而从*i*到*j*之间没有A的其他定值，那么，我们称*j*是四元式*i*的变量A的待用信息。（即下一个引用点）

i: A:=B op C

...

j: D:=A op E

- 假设在变量的符号表登记项中含有记录待用信息和活跃信息的栏。

待用信息和活跃信息的表示

- 1 (x, x)表示变量的待用信息和活跃信息。其中x表示待用信息，y表示活跃，^表示非待用和非活跃；
- 2 在符号表中，(x, x)→(x, x)表示后面的符号对代替前面的符号对；
- 3 不特别说明，所有说明变量在基本块出口之后均为非活跃变量。

- 计算待用信息和活跃信息的算法步骤：

1. 开始时，把基本块中各变量的符号表登记项中的待用信息栏填为“非待用”，并根据该变量在基本块出口之后是不是活跃的，把其中的活跃信息栏填为“活跃”或“非活跃”；
2. 从基本块出口到基本块入口由后向前依次处理各个四元式。对每一个四元式 i: A:=B op C，依次执行下面的步骤：
 - 1) 把符号表中变量A的待用信息和活跃信息附加到四元式i上；
 - 2) 把符号表中A的待用信息和活跃信息分别置为“非待用”和“非活跃”；
 - 3) 把符号表中变量B和C的待用信息和活跃信息附加到四元式i上；
 - 4) 把符号表中B和C的待用信息均置为i 活跃信息均置为“活跃”。

寄存器描述和地址描述

- 寄存器描述数组 RVALUE register
 - 动态记录各寄存器的使用信息
 - 例: $RVALUE[R] = \{A, B\}$
- 变量地址描述数组 AVALUE address
 - 动态记录各变量现行值的存放位置
 - 例: $AVALUE[A] = \{R1, R2, A\}$

补充说明

- 因为寄存器的分配是局限于基本块范围之内的，一旦处理完基本块中所有四元式，对现行值在寄存器中的每个变量，如果它在基本块之后是活跃的，则要把它存在寄存器中的值存放到它的主存单元中。
- 要特别强调的是，对形如: $A := B$ 的四元式，如果B的现行值在某寄存器 R_i 中，则无须生成目标代码，只须在 $RVALUE(R_i)$ 中增加一个A，(即把 R_i 同时分配给B和A)，并把 $AVALUE(A)$ 改为 R_i 。

代码生成算法

- 对每个四元式: $i: A := B \text{ op } C$ ，依次执行：
 1. 以四元式: $i: A := B \text{ op } C$ 为参数
调用函数过程 $GETREG(i: A := B \text{ op } C)$
返回一个寄存器R，用作存放A的寄存器。
 2. 利用 $AVALUE[B]$ 和 $AVALUE[C]$
确定B和C现行值的存放位置 B' 和 C' 。
如果其现行值在寄存器中，则把寄存器取作 B' 和 C'

3. 如果 $B' \neq R$, 则生成目标代码:

LD R, B'

op R, C'

否则生成目标代码 op R, C'

如果 B' 或 C' 为 R, 则删除 AVALUE[B] 或 AVALUE[C] 中的 R。

4. 令 $AVALUE[A] = \{R\}$, $RVALUE[R] = \{A\}$ 。

5. 若 B 或 C 的现行值在基本块中不再被引用, 也不是基本块出口之后的活跃变量, 且其现行值在某寄存器 R_k 中, 则删除 $RVALUE[R_k]$ 中的 B 或 C 以及 $AVALUE[B]$ 或 $AVALUE[C]$ 中的 R_k , 使得该寄存器不再为 B 或 C 占用。

■ 寄存器分配: GETREG(i: $A := B \text{ op } C$) 返回一个用来存放 A 的值的寄存器

1 尽可能用 B 独占的寄存器

2 尽可能用空闲寄存器

3 抢占用非空闲寄存器

1 如果 B 的现行值在某个寄存器 R_i 中, $RVALUE[R_i]$ 中只包含 B, 此外, 或者 B 与 A 是同一个标识符, 或者 B 的现行值在执行四元式 $A := B \text{ op } C$ 之后不会再引用, 则选取 R_i 为所需要的寄存器 R, 并转 4;

2 如果有尚未分配的寄存器, 则从中选取一个 R_i 为所需要的寄存器 R, 并转 4;

1 尽可能用 B 独占的寄存器

2 尽可能用空闲寄存器

3 抢占用非空闲寄存器

3 从已分配的寄存器中选取一个 R_i 为所需要的寄存器 R。最好使得 R_i 满足以下条件: 占用 R_i 的变量的值也同时存放在该变量的贮存单元中, 或者在基本块中要在最近的将来才会引用到或不会引用到。

4. 要不要为 R_i 中的变量 V 生成存数指令?

(1) 如果 V 的地址描述数组 AVALUE[V] 说 V 还保存在 R 之外的其他地方, 则不需要生成存数指令;

(2) 如果 V 是 A, 且不是 B 或 C, 则不需要生成存数指令;

(3) 如果 V 不会在此之后被使用, 则不需要生成存数指令;

(4) 否则, 生成目标代码 ST R_i , V

十一章的考点是：待用信息和活跃信息的表示+寄存器描述和地址描述+简单目标代码生成

示例 基本块

1. $T := A - B$
2. $U := A - C$
3. $V := T + U$
4. $W := V + U$

设W是基本块出口之后的活跃变量。

建立待用信息链表与活跃变量信息链表如下：

附加在四元式上的待用/活跃信息表：

序号	四元式	左值	左操作数	右操作数
(4)	$W := V + U$	$(\wedge, y)$	$(\wedge, \wedge)$	$(\wedge, \wedge)$
(3)	$V := T + U$	$(4, y)$	$(\wedge, \wedge)$	$(4, y)$
(2)	$U := A - C$	$(3, y)$	$(\wedge, \wedge)$	$(\wedge, \wedge)$
(1)	$T := A - B$	$(3, y)$	$(2, y)$	$(\wedge, \wedge)$
变量名 初始状态—信息链(待用/活跃信息链)				
T	$(\wedge, \wedge) \rightarrow (3, y) \rightarrow (\wedge, \wedge)$			
A	$(\wedge, \wedge) \rightarrow (2, y) \rightarrow (1, y)$			
B	$(\wedge, \wedge) \rightarrow (1, y)$			
C	$(\wedge, \wedge) \rightarrow (2, y)$			
U	$(\wedge, \wedge) \rightarrow (4, y) \rightarrow (3, y) \rightarrow (\wedge, \wedge)$			
V	$(\wedge, \wedge) \rightarrow (4, y) \rightarrow (\wedge, \wedge)$			
W	$(\wedge, y) \rightarrow (\wedge, \wedge)$			

中间代码

目标代码

RVALUE

AVALUE

$T := A - B$

LD R_0, A
SUB R_0, B

R_0 含有T

T在 R_0 中

$U := A - C$

LD R_1, A
SUB R_1, C

R_0 含有T
 R_1 含有U

T在 R_0 中
U在 R_1 中

$V := T + U$

ADD R_0, R_1

R_0 含有V
 R_1 含有U

V在 R_0 中
U在 R_1 中

$W := V + U$

ADD R_0, R_1
ST R_0, W

R_0 含有W

W在 R_0 中

例题：

示例 基本块

1. $T := A - B$
2. $U := A - C$
3. $V := T + U$
4. $W := V + U$
5. $Y := U - T$

■ 假设可用寄存器为 R_0 和 R_1 ，设W和Y是基本块出口的活跃变量

四元式	左值	左操作数	右操作数
$T := A - B$	$(3, Y)$	$(2, Y)$	$(\wedge, \wedge)$
$U := A - C$	$(3, Y)$	$(\wedge, \wedge)$	$(\wedge, \wedge)$
$V := T + U$	$(4, Y)$	$(5, \wedge)$	$(4, Y)$
$W := V + U$	$(\wedge, Y)$	$(\wedge, \wedge)$	$(5, Y)$
$Y := U - T$	$(\wedge, Y)$	$(\wedge, \wedge)$	$(\wedge, \wedge)$

中间代码

目标代码

RVALUE

AVALUE

$T := A - B$

LD R_0, A
SUB R_0, B

R_0 含有T

T在 R_0 中

$U := A - C$

LD R_1, A
SUB R_1, C

R_0 含有T
 R_1 含有U

T在 R_0 中
U在 R_1 中

$V := T + U$

ADD R_0, R_1

R_0 含有V
 R_1 含有U

V在 R_0 中
U在 R_1 中

$W := V + U$

ADD R_0, R_1
ST R_0, W

R_0 含有W

W在 R_0 中

第十一章复习完后自测：



第十一章-一道大题

- 目标代码生成的基本问题所涉及的基本概念
- 考虑寄存器分配的目标代码生成
- 待用与活跃信息表的构建